

DP Pattern Cheat Sheet

Generated 12/09/2026, 14:23:59 • 80 messages • Source: Shared ChatGPT link
<https://chatgpt.com/share/6aa512d2-84d8-83e8-91d1-3373c2553a17>

USER

explain.

ASSISTANT

This image is a **Dynamic Programming (DP) pattern cheat sheet**. It groups DP problems by the shape of their state and recurrence.

1. Prefix DP

State: Answer for the first i elements.

$$dp[i] = \text{answer using first } i \text{ items}$$

You build the solution from left to right.

Examples:

- Coin Change
- Climbing Stairs
- House Robber
- Decode Ways

Example: Climbing Stairs

$$dp[i] = dp[i-1] + dp[i-2]$$

To reach stair i , you come from either $i-1$ or $i-2$.

2. Sorted Prefix DP

First sort the input, then use prefix DP.

$$dp[i] = \text{best answer ending at sorted element } i$$

Sorting creates useful ordering relationships.

Examples:

- Longest Increasing Subsequence (LIS)
- Box Stacking
- Job Scheduling
- Longest Chain of Pairs

Example: LIS

After sorting isn't needed for standard LIS, but many related problems use ordered elements.

$$dp[i] = 1 + \max(dp[j])$$

for all valid previous positions j .

3. Two Sequences DP

State depends on prefixes of **two inputs**.

$\backslash dp[i][j] \backslash$

means answer using:

- first **i** characters of string A
- first **j** characters of string B

Examples:

- Longest Common Subsequence (LCS)
- Edit Distance

Example: LCS

If characters match:

$\backslash dp[i][j] = dp[i-1][j-1] + 1 \backslash$

Otherwise:

$\backslash dp[i][j] = \max(dp[i-1][j], dp[i][j-1]) \backslash$

4. Interval DP

State is a contiguous range.

$\backslash dp[i][j] \backslash$

means answer for subarray/subsequence from **i** to **j**.

Examples:

- Matrix Chain Multiplication
- Burst Balloons
- Palindrome Problems

Example: Matrix Chain Multiplication

$\backslash dp[i][j] = \min_{\{k\}} (dp[i][k] + dp[k+1][j] + \text{cost}) \backslash$

You try every split point **k**.

5. Grid DP

State corresponds to a cell in a 2D grid.

$\backslash dp[r][c] \backslash$

means answer up to position **(r, c)**.

Examples:

- Unique Paths
- Minimum Path Sum
- Maximal Square

Example: Unique Paths

$\backslash$ $dp[r][c] = dp[r-1][c] + dp[r][c-1]$ $\backslash$

because you can arrive from above or left.

How to recognize which pattern to use

Question asks about...	DP Pattern
First i elements	Prefix
Sorted items + chains	Sorted Prefix
Two strings/arrays	Two Sequences
Subarray [i..j]	Interval
Matrix/Grid traversal	Grid

Quick examples

- **House Robber** !' Prefix DP
- **Longest Common Subsequence** !' Two Sequences DP
- **Edit Distance** !' Two Sequences DP
- **Burst Balloons** !' Interval DP
- **Unique Paths** !' Grid DP
- **Minimum Path Sum** !' Grid DP

A useful rule of thumb:

- State looks like `dp[i]` !' Prefix DP
- State looks like `dp[i][j]` for two strings !' Two Sequences DP
- State looks like `dp[l][r]` on one array/string !' Interval DP
- State looks like `dp[row][col]` !' Grid DP

These five patterns cover a large majority of DP problems encountered in interviews and competitive programming.

USER

how to classify the problem ?

ASSISTANT

The easiest way to classify a DP problem is to ask:

"What does my state represent?"

The state usually tells you the pattern immediately.

1. Prefix DP

Ask:

"Can I define the answer using only the first i elements?"

State:

PYTHON

```
dp[i]
```

means:

TEXT

answer using first i elements

Examples:

- Climbing Stairs
- House Robber
- Decode Ways
- Coin Change

Clue

You are processing an array/string from left to right.

Example:

PYTHON

```
dp[i] = max(dp[i-1], dp[i-2] + nums[i])
```

(House Robber)

2. Two Sequences DP

Ask:

"Do I have two strings/arrays and need to compare them?"

State:

PYTHON

```
dp[i][j]
```

means:

TEXT

```
first i chars of A  
first j chars of B
```

Examples:

- LCS
- Edit Distance
- Distinct Subsequences

Clue

Problem statement contains:

TEXT

```
string A
string B
```

or

TEXT

```
array A
array B
```

and asks about matching, transforming, comparing.

3. Interval DP

Ask:

"Am I solving a subarray/subsequence [i,j]?"

State:

PYTHON

```
dp[i][j]
```

means:

TEXT

```
answer for interval [i,j]
```

Examples:

- Burst Balloons
- Matrix Chain Multiplication
- Palindrome Partitioning

Clue

You keep splitting ranges.

Typical recurrence:

PYTHON

```
for k in range(i, j):
    dp[i][j] = ...
```

If you see

TEXT

choose a partition point

or

TEXT

split the array

think Interval DP.

4. Grid DP

Ask:

"Is the input a matrix?"

State:

PYTHON

`dp[r][c]`

means:

TEXT

answer at cell (r,c)

Examples:

- Unique Paths
- Minimum Path Sum
- Maximal Square

Clue

Movement rules:

TEXT

up
down
left
right
diagonal

usually indicate Grid DP.

5. Sorted Prefix DP

Ask:

"Does sorting create useful order?"

Examples:

- LIS variants
- Russian Doll Envelopes
- Box Stacking

State:

```
PYTHON
```

```
dp[i]
```

means:

```
TEXT
```

```
best answer ending at i
```

after sorting.

Clue

The recurrence compares earlier elements:

```
PYTHON
```

```
for j in range(i):  
    if nums[j] < nums[i]:  
        ...
```

A practical interview flow

When you see a problem:

Step 1

How many changing parameters?

```
PYTHON
```

```
dp[i]
```

!' Prefix DP candidate

```
PYTHON
```

```
dp[i][j]
```

!' Two Sequences or Interval

Step 2

What do i and j mean?

If:

TEXT

```
i in string1
j in string2
```

!' Two Sequences

If:

TEXT

```
i = left boundary
j = right boundary
```

!' Interval

If:

TEXT

```
i = row
j = column
```

!' Grid

Step 3

Can answer be built from previous positions?

PYTHON

```
dp[i]
```

using

PYTHON

```
dp[i-1]
dp[i-2]
...
```

!' Prefix DP

Examples

Problem	Pattern
House Robber	Prefix
Climbing Stairs	Prefix
Coin Change	Prefix
Longest Common Subsequence	Two Sequences
Edit Distance	Two Sequences
Distinct Subsequences	Two Sequences
Burst Balloons	Interval

Problem	Pattern
Matrix Chain Multiplication	Interval
Longest Palindromic Subsequence	Interval
Unique Paths	Grid
Minimum Path Sum	Grid
Maximal Square	Grid
LIS	Sorted Prefix

A very reliable shortcut:

- **One string/array !** usually Prefix DP.
- **Two strings/arrays !** usually Two Sequences DP.
- **Matrix input !** usually Grid DP.
- **Subarray [i,j] matters !** usually Interval DP.
- **Need sorting before DP !** usually Sorted Prefix DP.

USER

how to solve any prefix type problems ???

ASSISTANT

You can't solve **every** Prefix DP problem with one formula, but you can solve almost all of them with the same thought process.

Step 1: Define `dp[i]`

Always start with:

What does `dp[i]` represent?

For Prefix DP:

TEXT

`dp[i]` = answer using the first `i` elements

or

TEXT

`dp[i]` = best answer ending at position `i`

Example: Climbing Stairs

TEXT

`dp[i] = ways to reach stair i`

Now ask:

How can I get to stair i?

Only from:

TEXT

`i - 1`
`i - 2`

Therefore

PYTHON

`dp[i] = dp[i-1] + dp[i-2]`

Step 2: Find the last action

This is the most important trick.

Ask:

What was the last thing I did before reaching state i?

Example: House Robber

Suppose we are at house `i`.

The last decision is:

Option 1

Don't rob house i

PYTHON

`dp[i-1]`

Option 2

Rob house i

PYTHON

`dp[i-2] + nums[i]`

Take best:

PYTHON

```
dp[i] = max(  
    dp[i-1],  
    dp[i-2] + nums[i]  
)
```

Step 3: Write recurrence

Almost every Prefix DP becomes

PYTHON

```
dp[i] = best(previous states)
```

Common forms:

Maximum

PYTHON

```
dp[i] = max(...)
```

Minimum

PYTHON

```
dp[i] = min(...)
```

Counting

PYTHON

```
dp[i] = sum(...)
```

Step 4: Base cases

Ask:

What are the smallest valid prefixes?

Example:

PYTHON

```
dp[0]  
dp[1]
```

These are usually obvious.

Generic Prefix DP Template

PYTHON

```
dp = [0] * n

initialize base cases

for i in range(start, n):
    dp[i] = recurrence

return dp[n-1]
```

How to derive recurrence quickly

Suppose you're stuck.

For index `i`, ask:

Question 1

Can I come from `i-1`?

Question 2

Can I come from `i-2`?

Question 3

Can I come from any earlier index?

PYTHON

```
for j in range(i):
```

If yes, recurrence usually becomes:

PYTHON

```
dp[i] = max(
    dp[j] + something
    for j < i
)
```

This pattern appears in:

- LIS
- Word Break
- Jump Game variants

Example: Coin Change

State:

TEXT

```
dp[i] = minimum coins needed to make amount i
```

Last action:

Which coin was used last?

If coin = c

PYTHON

```
dp[i] = dp[i-c] + 1
```

Try all coins:

PYTHON

```
dp[i] = min(  
    dp[i-c] + 1  
)
```

Example: Decode Ways

State:

TEXT

```
dp[i] = ways to decode first i characters
```

Last action:

Either:

TEXT

```
take one digit
```

or

TEXT

```
take two digits
```

Therefore:

PYTHON

```
dp[i] += dp[i-1]  
dp[i] += dp[i-2]
```

when valid.

A universal checklist

For a 1D DP problem:

1. Define

TEXT

```
dp[i]
```

2. Ask

TEXT

What is the last action?

3. Determine where you can come from

TEXT

i-1 ?
i-2 ?
any j < i ?

4. Write recurrence

PYTHON

```
dp[i] = combine(previous states)
```

5. Add base cases

PYTHON

```
dp[0]  
dp[1]
```

If you consistently think in terms of **"What does `dp[i]` mean?"** and **"What was the last action?"**, you'll be able to derive most Prefix DP recurrences without memorizing them.

USER

A string consisting of uppercase english characters can be encoded to a number using the following mapping:

'A' -> "1" 'B' -> "2" ... 'Z' -> "26" To decode a message, digits must be grouped and then mapped back into letters using the reverse of the mapping above. There may be multiple ways to decode a message. For example, "1012" can be mapped into:

"JAB" with the grouping (10 1 2) "JL" with the grouping (10 12) The grouping (1 01 2) is invalid because 01 cannot be mapped into a letter since it contains a leading zero.

Given a string s containing only digits, return the number of ways to decode it. You can assume that the answer fits in a 32-bit integer.

Example 1:

Input: s = "12"

Output: 2

Explanation: "12" could be decoded as "AB" (1 2) or "L" (12). Example 2:

Input: s = "01"

Output: 0 Explanation: "01" cannot be decoded because "01" cannot be mapped into a letter.

Constraints:

1 <= s.length <= 100 s consists of digits

show the though process and go to the solution

ASSISTANT

For DP problems, it's much more useful to learn **how to discover the state and recurrence** than to memorize the solution.

Let's walk through this one step by step.

Step 1: What are we trying to count?

Given:

TEXT

"12"

We want:

TEXT

AB = 1 2
L = 12

Answer = 2.

Notice we're counting **ways**, not finding a minimum or maximum.

That usually suggests:

TEXT

dp[i] = number of ways

Step 2: What should the state be?

This is a Prefix DP problem.

Define:

TEXT

dp[i] = number of ways to decode the first i characters

For example:

TEXT

s = "121"

dp[0] = ways to decode ""
dp[1] = ways to decode "1"
dp[2] = ways to decode "12"
dp[3] = ways to decode "121"

Why use "first i characters"?

Because once we know how many ways smaller prefixes can be decoded, we can extend them.

Step 3: Look at the LAST decoding choice

This is the key DP trick.

Suppose we're computing:

```
TEXT
dp[i]
```

How could the decoding end?

There are only two possibilities:

Case 1: Last letter uses one digit

Example:

```
TEXT
121
  ^
```

Last digit:

```
TEXT
1 -> A
```

Then everything before it must already be decoded.

So:

```
TEXT
ways += dp[i-1]
```

Case 2: Last letter uses two digits

Example:

```
TEXT
121
 ^^
21 -> U
```

Then everything before those two digits must already be decoded.

So:

```
TEXT
ways += dp[i-2]
```

Step 4: When are these cases valid?

Single digit

A single digit can represent:

```
TEXT
1..9
```

but NOT:

```
TEXT
0
```

So:

```
PYTHON
if s[i-1] != '0':
    dp[i] += dp[i-1]
```

Two digits

The two-digit number must be:

```
TEXT
10..26
```

because:

```
TEXT
10 -> J
...
26 -> Z
```

Anything else is invalid.

So:

```
PYTHON
two = int(s[i-2:i])

if 10 <= two <= 26:
    dp[i] += dp[i-2]
```

Step 5: Base cases

Empty string

There is exactly one way to decode nothing:

```
TEXT
do nothing
```

So:

PYTHON

```
dp[0] = 1
```

First character

If:

TEXT

"0"

there are no ways.

Otherwise:

PYTHON

```
dp[1] = 1
```

Example

s = "12"

Length = 2

Initialize:

TEXT

```
dp = [1, 1, 0]
```

i = 2

Single digit:

TEXT

2

valid

TEXT

```
dp[2] += dp[1]  
      += 1
```

Now:

TEXT

```
dp[2] = 1
```

Two digits:

TEXT

12

valid

TEXT

```
dp[2] += dp[0]
      += 1
```

Now:

TEXT

dp[2] = 2

Answer:

TEXT

2

Example

s = "1012"

dp[0]

TEXT

1

dp[1]

TEXT

1

i = 2

TEXT

10

Single digit:

TEXT

0

invalid

Two digits:

TEXT

10

valid

TEXT

$dp[2] = dp[0] = 1$

i = 3

TEXT

1

valid

TEXT

$dp[3] += dp[2]$

Two digits:

TEXT

01

invalid

TEXT

$dp[3] = 1$

i = 4

TEXT

2

valid

TEXT

$dp[4] += dp[3]$

Two digits:

TEXT

12

valid

TEXT

$dp[4] += dp[2]$

Result:

TEXT

$dp[4] = 2$

which corresponds to:

TEXT

10 1 2

10 12

Recurrence

$$dp[i] = \begin{cases} dp[i] + dp[i-1] & \text{if last digit is } 1..9 \\ dp[i] + dp[i-2] & \text{if last two digits are } 10..26 \end{cases}$$

This is the entire DP.

Code

PYTHON

```
class Solution:
    def numDecodings(self, s: str) -> int:
        n = len(s)

        if s[0] == '0':
            return 0

        dp = [0] * (n + 1)

        dp[0] = 1
        dp[1] = 1

        for i in range(2, n + 1):
            if s[i - 1] != '0':
                dp[i] += dp[i - 1]

            two = int(s[i - 2:i])
            if 10 <= two <= 26:
                dp[i] += dp[i - 2]

        return dp[n]
```

The important insight is not the code itself, but the observation:

To decode the first i characters, the last decoded letter must come from either the last 1 digit or the last 2 digits.

Once you see that, the recurrence almost writes itself.

USER

. Interval DP

State is a contiguous range. what do you mean ???

ASSISTANT

A **contiguous range** means a segment where **everything between the endpoints is included**.

For an array:

TEXT

Index: 0 1 2 3 4
Array: a b c d e

Examples of contiguous ranges:

TEXT

[a]
[b c]
[c d e]
[a b c d]

Not contiguous:

TEXT

[a c]
[b d e]

because elements in the middle are skipped.

What does `dp[i][j]` mean in Interval DP?

In Interval DP:

TEXT

`dp[i][j]`

means

TEXT

answer for the subarray/subsequence
from index i to index j

For example:

TEXT

Array = [3, 1, 5, 8]

Then:

TEXT

`dp[1][2]`

represents the answer for:

TEXT

```
[1, 5]
```

and

TEXT

```
dp[0][3]
```

represents the answer for:

TEXT

```
[3, 1, 5, 8]
```

Why can't Prefix DP work?

In Prefix DP, state is:

TEXT

```
dp[i]
```

which only represents:

TEXT

```
first i elements
```

But some problems ask:

TEXT

```
"What is the best answer inside this range?"
```

or

TEXT

```
"What happens if I split this interval?"
```

Then one index is not enough.

Example 1: Longest Palindromic Subsequence

String:

TEXT

```
"bbbab"
```

Suppose we define:

TEXT

```
dp[i][j]
=
length of longest palindromic subsequence
inside s[i...j]
```

Now look at:

TEXT

```
b b b a b
0 1 2 3 4
```

To compute:

TEXT

```
dp[0][4]
```

we need answers for smaller intervals:

TEXT

```
dp[1][4]
dp[0][3]
dp[1][3]
```

Notice how everything is about ranges.

Example 2: Matrix Chain Multiplication

Suppose:

TEXT

```
A1 A2 A3 A4
```

We want the cheapest way to multiply them.

State:

TEXT

```
dp[i][j]
=
minimum cost to multiply matrices
Ai ... Aj
```

To solve:

TEXT

```
dp[1][4]
```

we try every split:

TEXT

(A1) (A2A3A4)

(A1A2) (A3A4)

(A1A2A3) (A4)

Graphically:

TEXT

i j
|-----|
 k

Recurrence:

PYTHON

```
for k in range(i, j):  
    dp[i][j] = min(  
        dp[i][k] +  
        dp[k+1][j] +  
        merge_cost  
    )
```

This "try every split point k" is a huge clue that the problem is Interval DP.

Example 3: Burst Balloons

Array:

TEXT

[3,1,5,8]

State:

TEXT

dp[i][j]

means

TEXT

maximum coins obtainable
from balloons i to j

Again we solve smaller intervals and combine them.

How to recognize Interval DP quickly

If you see phrases like:

TEXT

subarray
substring
range
interval
segment

or

TEXT

choose a split point
partition
cut
divide

think:

TEXT

`dp[i][j]`

Comparison

House Robber

TEXT

`[2, 7, 9, 3, 1]`

Only care about:

TEXT

first i houses

So:

TEXT

`dp[i]`

!' Prefix DP

Palindrome Problem

TEXT

"abca"

Need answer for:

TEXT

`s[i...j]`

So:

TEXT

`dp[i][j]`

!' Interval DP

A useful rule:

If the state naturally sounds like "answer for the range from i to j", it's almost certainly Interval DP. If it sounds like "answer for the first i elements", it's Prefix DP.

USER

longest palindromic subsequence , categorise this problem and show the thought process in solving this.

ASSISTANT

This is one of the best problems for learning **Interval DP** because Prefix DP fails almost immediately.

Problem

Given:

TEXT

`s = "bbbab"`

Find the length of the **longest palindromic subsequence**.

Subsequence means you can skip characters.

For example:

TEXT

`bbbb`

is a palindrome subsequence of `"bbbab"`.

Answer:

TEXT

`4`

Step 1: Classify the problem

Ask:

What does the answer depend on?

Suppose I ask:

TEXT

Longest palindromic subsequence in:

"bbbab"

Can I define

TEXT

`dp[i]`

as the answer for the first `i` characters?

Not really.

Because the palindrome property depends on **both ends**.

Example:

TEXT

```
b b b a b
^       ^
```

Whether the first and last characters match matters.

So we need two indices.

Step 2: What should the state be?

Whenever a problem talks about:

TEXT

substring
subarray
range
`s[i...j]`

try:

TEXT

`dp[i][j]`

Define:

TEXT

```
dp[i][j]
=
length of longest palindromic subsequence
inside s[i...j]
```

Example:

TEXT

```
s = "bbbab"
```

TEXT

```
dp[1][3]
```

means:

TEXT

```
"bba"
```

What's the longest palindromic subsequence inside "bba" ?

Answer:

TEXT

```
2
```

("bb")

Step 3: Think about the ends

This is the key observation.

For interval DP, always look at:

TEXT

```
s[i]  
s[j]
```

the two ends of the interval.

Suppose:

TEXT

```
s[i] == s[j]
```

Example:

TEXT

```
b ... b
```

Then these two characters can be part of the palindrome.

So:

TEXT

```
answer =  
2 +  
best answer inside
```

Graphically:

```
TEXT
b [inside] b
```

Therefore:

```
PYTHON
dp[i][j] = 2 + dp[i+1][j-1]
```

Example

```
TEXT
"bab"
```

```
TEXT
b a b
```

Ends match.

```
TEXT
2 + dp[1][1]
```

and

```
TEXT
dp[1][1] = 1
```

So:

```
TEXT
3
```

Correct.

Step 4: What if ends don't match?

Example:

```
TEXT
b a c
```

```
TEXT
^   ^
```

They cannot both be used.

At least one must be discarded.

Two choices:

Remove left

TEXT

a c

which is:

PYTHON

```
dp[i+1][j]
```

Remove right

TEXT

b a

which is:

PYTHON

```
dp[i][j-1]
```

Take the better one:

PYTHON

```
dp[i][j] = max(  
    dp[i+1][j],  
    dp[i][j-1]  
)
```

Recurrence

If:

TEXT

```
s[i] == s[j]
```

then

PYTHON

```
dp[i][j] = 2 + dp[i+1][j-1]
```

Otherwise

PYTHON

```
dp[i][j] = max(  
    dp[i+1][j],  
    dp[i][j-1]  
)
```

Step 5: Base cases

Single character:

```
TEXT
"a"
```

Longest palindrome:

```
TEXT
"a"
```

length:

```
TEXT
1
```

Therefore:

```
PYTHON
dp[i][i] = 1
```

Step 6: Which order should we fill?

Notice:

```
PYTHON
dp[i][j]
```

depends on

```
PYTHON
dp[i+1][j]
dp[i][j-1]
dp[i+1][j-1]
```

These are **smaller intervals**.

So fill intervals from small to large.

Visualization

Length 1:

```
TEXT
dp[0][0]
dp[1][1]
dp[2][2]
...
```

Length 2:

```
TEXT
dp[0][1]
dp[1][2]
...
```

Length 3:

```
TEXT
dp[0][2]
dp[1][3]
...
```

and so on.

Example

```
TEXT
s = "bbbab"
```

Initialize diagonals:

```
TEXT
1 1 1 1 1
```

Now length 2 intervals:

```
TEXT
"bb" -> 2
"bb" -> 2
"ba" -> 1
"ab" -> 1
```

Length 3:

```
TEXT
"bbb" -> 3
"bba" -> 2
"bab" -> 3
```

Length 4:

```
TEXT
"bbba" -> 3
"bbab" -> 3
```

Length 5:

```
TEXT
"bbbab" -> 4
```

Answer:

TEXT

4

Complete code

PYTHON

```
class Solution:
    def longestPalindromeSubseq(self, s: str) -> int:
        n = len(s)

        dp = [[0] * n for _ in range(n)]

        for i in range(n):
            dp[i][i] = 1

        for length in range(2, n + 1):
            for i in range(n - length + 1):
                j = i + length - 1

                if s[i] == s[j]:
                    if length == 2:
                        dp[i][j] = 2
                    else:
                        dp[i][j] = 2 + dp[i + 1][j - 1]
                else:
                    dp[i][j] = max(
                        dp[i + 1][j],
                        dp[i][j - 1]
                    )

        return dp[0][n - 1]
```

Pattern recognition takeaway

When you see:

- A string or array.
- Questions about a **substring/subarray**.
- Decisions based on the **left and right ends**.
- Recurrences involving `i+1`, `j-1`, or trying splits.

Think:

TEXT

```
dp[i][j] = answer for interval [i, j]
```

That's the hallmark of **Interval DP**.

USER

$dp[i][j] = 2 + dp[i+1][j-1]$ why $i + 1, j - 1$?

ASSISTANT

Because once you've decided to use **both ends** of the interval in the palindrome, those ends are already accounted for.

Let's visualize it.

Suppose:

TEXT

`s = "bab"`

Indices:

TEXT

```
0 1 2
b a b
^     ^
i     j
```

Since:

PYTHON

```
s[i] == s[j]
```

both `'b'` characters can be included in the palindrome.

That contributes:

TEXT

2

to the answer.

Now the only part left to solve is the **inside**:

TEXT

```
a
^
1
```

which is:

PYTHON

```
dp[i+1][j-1]
```

So:

PYTHON

```
dp[0][2] = 2 + dp[1][1]
```

Another example:

TEXT

```
s = "abcba"
```

TEXT

```
a b c b a
^         ^
i         j
```

The two 'a' s match.

Use them.

Now solve the remaining middle:

TEXT

```
b c b
```

which is:

PYTHON

```
dp[1][3]
```

Therefore:

PYTHON

```
dp[0][4] = 2 + dp[1][3]
```

Why not `dp[i][j-1]` ?

Because `dp[i][j-1]` still contains the left 'a' .

TEXT

```
a b c b
^
```

You've already decided to use that 'a' .

You shouldn't count it again.

Why not `dp[i+1][j]` ?

Same reason.

TEXT

```
b c b a
         ^
```

The right 'a' is still present.

You've already used it.

The intuition

When the ends match:

```
TEXT
s[i] == s[j]
```

you "lock them in" as part of the palindrome:

```
TEXT
s[i] + (best palindrome inside) + s[j]
```

Graphically:

```
TEXT
i                j
|                |
a  b  c  b  a
  |-----|
  i+1 j-1
```

So the remaining subproblem is exactly the interval:

```
PYTHON
[i+1, j-1]
```

which gives:

```
PYTHON
dp[i][j] = 2 + dp[i+1][j-1]
```

The 2 comes from the matching endpoints, and `dp[i+1][j-1]` is the longest palindromic subsequence inside those endpoints.

```
USER
consider a b c d a , there is no palidrome inside ,what happens now
```

ASSISTANT

Excellent question. This gets to the subtle part of the recurrence.

Consider:

```
TEXT
a b c d a
0 1 2 3 4
```

We have:

```
PYTHON
s[0] == s[4]
```

so the recurrence says:

```
PYTHON
dp[0][4] = 2 + dp[1][3]
```

Now look at the inside:

```
TEXT
b c d
1 2 3
```

There is no palindrome longer than 1 there.

Let's compute it:

```
TEXT
dp[1][1] = 1
dp[2][2] = 1
dp[3][3] = 1
```

For length 2:

```
TEXT
"bc" -> 1
"cd" -> 1
```

For length 3:

```
TEXT
"bcd" -> 1
```

Thus:

```
PYTHON
dp[1][3] = 1
```

Therefore:

```
PYTHON
dp[0][4] = 2 + 1 = 3
```

And that's correct!

The longest palindromic subsequence is:

```
TEXT
a c a
```

or

TEXT

a b a

or

TEXT

a d a

all of length 3.

Why does adding 2 still work?

Because `dp[i+1][j-1]` is not asking for a palindrome that uses the entire interval.

It's asking:

What is the longest palindromic subsequence anywhere inside?

For:

TEXT

b c d

the answer is:

TEXT

b

(length 1)

Then we wrap the matching 'a' s around it:

TEXT

a b a

giving length:

TEXT

2 + 1 = 3

Another example

TEXT

a b x y z a

Inside:

TEXT

b x y z

Longest palindromic subsequence inside is still just one character:

TEXT

1

So:

PYTHON

```
dp[0][5] = 2 + 1 = 3
```

One valid palindrome is:

TEXT

a b a

The important realization

Many people mistakenly think:

PYTHON

```
dp[i+1][j-1]
```

must be a palindrome that spans the whole interval.

It doesn't.

It means:

TEXT

Longest palindromic subsequence
contained somewhere inside $[i+1, j-1]$

Even if the inside looks like:

TEXT

b c d

the longest palindromic subsequence is still:

TEXT

b

(length 1), because every single character is a palindrome.

That's why the recurrence still works:

PYTHON

```
if s[i] == s[j]:  
    dp[i][j] = 2 + dp[i+1][j-1]
```

The inside answer is **never 0** unless the interval is empty. For any non-empty interval, there's always at least a palindrome of length 1.

USER

Longest Palindromic Substring Medium Topics Company Tags Hints Given a string *s*, return the longest substring of *s* that is a palindrome.

A palindrome is a string that reads the same forward and backward.

If there are multiple palindromic substrings that have the same length, return any one of them.
the question is actually substring

ASSISTANT

Yes, and that's exactly why **Longest Palindromic Subsequence (LPS)** and **Longest Palindromic Substring (LPSUB)** are different problems with different recurrences.

Subsequence vs Substring

Subsequence

You can skip characters.

TEXT

a b c d a

can form:

TEXT

a c a

because skipping is allowed.

That's why for Longest Palindromic Subsequence we used:

PYTHON

```
dp[i][j] = 2 + dp[i+1][j-1]
```

when ends match.

Substring

You cannot skip characters.

The characters must be contiguous.

Example:

TEXT

a b c d a

Possible substrings:

TEXT

ab
bcd
cda
abcda

Not allowed:

TEXT

aca

because the characters are not contiguous.

Why the old recurrence fails

Consider:

TEXT

a b c d a

For subsequence:

PYTHON

```
dp[0][4] = 2 + dp[1][3]
```

gave:

TEXT

a b a

length 3.

But:

TEXT

aba

is NOT a substring of "abcda".

So that recurrence cannot be used.

How to think about Longest Palindromic Substring

Define:

PYTHON

```
dp[i][j]
```

as:

TEXT

Is `s[i...j]` a palindrome?

Notice:

TEXT

Boolean state

not length.

Base cases

Length 1:

TEXT

a
b
c

are palindromes.

PYTHON

```
dp[i][i] = True
```

Length 2:

TEXT

aa
bb
cc

are palindromes.

PYTHON

```
dp[i][i+1] = (s[i] == s[i+1])
```

General case

For:

TEXT

`s[i...j]`

to be a palindrome:

1. Ends must match

PYTHON

```
s[i] == s[j]
```

2. Middle must already be a palindrome

PYTHON

```
dp[i+1][j-1]
```

Therefore:

PYTHON

```
dp[i][j] = (  
    s[i] == s[j]  
    and  
    dp[i+1][j-1]  
)
```

Example

TEXT

babad

Check:

TEXT

bab

Ends:

TEXT

b == b

Middle:

TEXT

a

is palindrome.

Therefore:

TEXT

bab

is palindrome.

Why this is still Interval DP

State:

PYTHON

```
dp[i][j]
```

represents an interval:

TEXT

```
s[i...j]
```

and depends on:

PYTHON

```
dp[i+1][j-1]
```

which is a smaller interval.

So it is still **Interval DP**.

Pattern recognition

Longest Palindromic Subsequence

State:

PYTHON

```
dp[i][j]
```

means:

TEXT

```
length of longest palindromic subsequence  
inside interval [i,j]
```

Choices:

TEXT

```
skip left  
skip right  
take both
```

Longest Palindromic Substring

State:

PYTHON

```
dp[i][j]
```

means:

TEXT

```
is interval [i,j] itself a palindrome?
```

No skipping allowed.

Recurrence:

PYTHON

```
dp[i][j] = (
    s[i] == s[j]
    and
    dp[i+1][j-1]
)
```

A useful interview rule:

If the problem says substring, think "contiguous interval" and often use a boolean `dp[i][j]` that describes the interval itself.

If the problem says **subsequence**, think "I may skip characters", and the recurrence often involves taking max/min over smaller intervals.

USER

now the full code

ASSISTANT

Here's the classic $O(n^2)$ DP solution for Longest Palindromic Substring:

PYTHON

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        n = len(s)

        dp = [[False] * n for _ in range(n)]

        start = 0
        max_len = 1

        # Length 1 substrings
        for i in range(n):
            dp[i][i] = True

        # Length 2 substrings
        for i in range(n - 1):
            if s[i] == s[i + 1]:
                dp[i][i + 1] = True
                start = i
                max_len = 2

        # Length >= 3
        for length in range(3, n + 1):
            for i in range(n - length + 1):

                j = i + length - 1

                if s[i] == s[j] and dp[i + 1][j - 1]:
                    dp[i][j] = True
```

```
        if length > max_len:
            max_len = length
            start = i

    return s[start:start + max_len]
```

Filling order

For:

```
PYTHON
dp[i][j]
```

we need:

```
PYTHON
dp[i + 1][j - 1]
```

already computed.

So we fill intervals by increasing length:

```
TEXT
Length 1
[a]
[b]
[c]

Length 2
[ab]
[bc]

Length 3
[aba]
[bcb]

Length 4
[abba]
```

This is the standard Interval DP filling order.

Example

```
PYTHON
s = "babad"
```

Length 1:

```
TEXT
b a b a d
```

All are palindromes.

Length 2:

TEXT

ba
ab
ba
ad

None are palindromes.

Length 3:

TEXT

bab

Ends match and middle "a" is palindrome.

So:

PYTHON

```
dp[0][2] = True
```

Similarly:

TEXT

aba

is palindrome.

Answer becomes either:

TEXT

bab

or

TEXT

aba

both are accepted.

Space Optimization

For interviews, there is an even cleaner **expand around center** solution:

PYTHON

```
class Solution:
    def longestPalindrome(self, s: str) -> str:

        ans = ""

        def expand(l, r):
            while l >= 0 and r < len(s) and s[l] == s[r]:
                l -= 1
                r += 1
```

```

        return s[l + 1:r]

    for i in range(len(s)):

        p1 = expand(i, i)      # odd length
        p2 = expand(i, i + 1) # even length

        if len(p1) > len(ans):
            ans = p1

        if len(p2) > len(ans):
            ans = p2

    return ans

```

This is also $O(n^2)$ time but only $O(1)$ extra space, and it's the solution most interviewers expect.

USER

```

for length in range(3, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        if s[i] == s[j] and dp[i + 1][j - 1]: dp[i][j] = True
        if length > max_len: max_len = length start = i

```

explain this part

ASSISTANT

Let's focus on just this loop.

PYTHON

```

for length in range(3, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        if s[i] == s[j] and dp[i + 1][j - 1]:
            dp[i][j] = True

        if length > max_len:
            max_len = length
            start = i

```

Suppose:

PYTHON

```
s = "babad"
```

Indices:

TEXT

```
0 1 2 3 4
b a b a d
```

What does `length` mean?

We're checking all substrings of a particular length.

`length = 3`

Possible substrings:

TEXT

```
bab
aba
bad
```

`length = 4`

Possible substrings:

TEXT

```
baba
abad
```

`length = 5`

Possible substrings:

TEXT

```
babad
```

What does `i` mean?

`i` is the starting index of the substring.

For:

PYTHON

```
length = 3
```

we have:

PYTHON

```
for i in range(n - length + 1):
```

PYTHON

```
range(5 - 3 + 1)
```

```
PYTHON
```

```
range(3)
```

SO:

```
TEXT
```

```
i = 0,1,2
```

These correspond to:

```
TEXT
```

```
0 -> "bab"
```

```
1 -> "aba"
```

```
2 -> "bad"
```

What is `j` ?

```
PYTHON
```

```
j = i + length - 1
```

`j` is the ending index.

Example:

```
PYTHON
```

```
i = 1
```

```
length = 3
```

Then:

```
PYTHON
```

```
j = 1 + 3 - 1  
  = 3
```

So we're examining:

```
TEXT
```

```
s[1:4]
```

which is:

```
TEXT
```

```
aba
```

Why check

PYTHON

```
s[i] == s[j]
```

?

For a palindrome:

TEXT

```
aba  
^  ^
```

first and last characters must match.

If they don't:

TEXT

```
abc  
^  ^
```

then it can never be a palindrome.

Why check

PYTHON

```
dp[i + 1][j - 1]
```

?

Remember:

PYTHON

```
dp[x][y]
```

means:

TEXT

```
Is s[x...y] a palindrome?
```

For:

TEXT

```
a b a  
^  ^
```

the outer characters match.

Now we need to know:

TEXT

```
b
```

is a palindrome.

That's exactly:

```
PYTHON
dp[1][1]
```

Example

Substring:

```
TEXT
abba
```

Indices:

```
TEXT
0 1 2 3
a b b a
```

We check:

```
PYTHON
s[0] == s[3]
```

True.

Then check:

```
PYTHON
dp[1][2]
```

which asks:

```
TEXT
Is "bb" a palindrome?
```

Yes.

Therefore:

```
PYTHON
dp[0][3] = True
```

meaning:

```
TEXT
"abba" is a palindrome
```

Why update `max_len`?

Whenever we find a palindrome:

```
PYTHON
```

```
dp[i][j] = True
```

we ask:

```
PYTHON
```

```
if length > max_len:
```

because we want the **longest** palindrome.

Example

Initially:

```
PYTHON
```

```
max_len = 1
```

because every character is a palindrome.

When we find:

```
TEXT
```

```
bab
```

length:

```
PYTHON
```

```
3
```

Then:

```
PYTHON
```

```
max_len = 3  
start = 0
```

Later we find:

```
TEXT
```

```
aba
```

length:

```
PYTHON
```

```
3
```

Since:

```
PYTHON
```

```
3 > 3
```

is false,

we keep:

```
TEXT
```

```
bab
```

as the answer.

Visualizing one iteration

Suppose:

```
TEXT
```

```
s = "babad"
```

and we're checking:

```
TEXT
```

```
aba
```

```
TEXT
```

```
0 1 2 3 4
b a b a d
  ^   ^
  i   j
```

```
PYTHON
```

```
s[i] == s[j]
```

```
TEXT
```

```
a == a
```

True.

Check:

```
PYTHON
```

```
dp[i+1][j-1]
```

```
PYTHON
```

```
dp[2][2]
```

which is:

TEXT

"b"

and every single character is a palindrome.

So:

PYTHON

```
dp[1][3] = True
```

meaning:

TEXT

"aba"

is a palindrome.

The most important line is:

PYTHON

```
if s[i] == s[j] and dp[i + 1][j - 1]:
```

which literally translates to:

A substring is a palindrome if its first and last characters match and the substring inside them is already known to be a palindrome.

USER

Palindromic Substrings Medium Topics Company Tags Hints Given a string *s*, return the number of substrings within *s* that are palindromes.

A palindrome is a string that reads the same forward and backward.

ASSISTANT

This is very closely related to **Longest Palindromic Substring**.

Step 1: Classify the problem

The problem asks about **substrings** (contiguous intervals).

So think:

TEXT

Interval DP

or even better:

TEXT

Expand Around Center

Step 2: What are we counting?

We need the number of palindromic substrings.

Example:

TEXT

s = "aaa"

Palindromic substrings:

TEXT

"a" (index 0)
"a" (index 1)
"a" (index 2)
"aa" (0-1)
"aa" (1-2)
"aaa" (0-2)

Answer:

TEXT

6

DP Thought Process

Define:

PYTHON

dp[i][j]

as

TEXT

Is substring s[i...j] a palindrome?

(Boolean state)

Base Case 1

Every single character is a palindrome.

PYTHON

dp[i][i] = True

Count them:

PYTHON

```
ans += 1
```

Base Case 2

Length 2 substrings:

PYTHON

```
if s[i] == s[i+1]:  
    dp[i][i+1] = True
```

Count them.

Length >= 3

A substring is a palindrome iff:

1. Ends match.
2. Inside is already a palindrome.

PYTHON

```
if s[i] == s[j] and dp[i+1][j-1]:  
    dp[i][j] = True
```

Whenever this happens:

PYTHON

```
ans += 1
```

Full DP Solution

PYTHON

```
class Solution:  
    def countSubstrings(self, s: str) -> int:  
        n = len(s)  
  
        dp = [[False] * n for _ in range(n)]  
  
        count = 0  
  
        # Length 1  
        for i in range(n):  
            dp[i][i] = True  
            count += 1  
  
        # Length 2  
        for i in range(n - 1):  
            if s[i] == s[i + 1]:  
                dp[i][i + 1] = True  
                count += 1  
  
        # Length >= 3
```

```

for length in range(3, n + 1):
    for i in range(n - length + 1):
        j = i + length - 1
        if s[i] == s[j] and dp[i + 1][j - 1]:
            dp[i][j] = True
            count += 1

return count

```

Time:

TEXT
 $O(n^2)$

Space:

TEXT
 $O(n^2)$

Better Solution: Expand Around Center

Notice that every palindrome has a center.

For:

TEXT
 aba

center is:

TEXT
 b

For:

TEXT
 abba

center is:

TEXT
 bb

So for every position:

- expand for odd length palindromes
- expand for even length palindromes

and count them.

PYTHON

```
class Solution:
    def countSubstrings(self, s: str) -> int:
        n = len(s)
        count = 0

        def expand(left, right):
            nonlocal count

            while left >= 0 and right < n and s[left] == s[right]:
                count += 1
                left -= 1
                right += 1

        for i in range(n):
            expand(i, i)          # odd length
            expand(i, i + 1)      # even length

        return count
```

Time:

TEXT

$O(n^2)$

Space:

TEXT

$O(1)$

For interviews, the **expand-around-center solution** is usually preferred because it's simpler and uses constant extra space.

USER

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        dp = [[0]*n for _ in range(n)]
        for i in range(n):
            dp[i][i] = 1
            for length in range(2, n + 1):
                for i in range(n - length + 1):
                    j = i + length - 1
                    if nums[i] > nums[j]:
                        if length == 2:
                            dp[i][j] = 2
                        else:
                            dp[i][j] = 2 + dp[i + 1][j - 1]
                    else:
                        dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])

        return dp[0][n - 1]
```

ASSISTANT

Your code is actually the recurrence for **Longest Palindromic Subsequence**, not **Longest Increasing Subsequence (LIS)**.

You seem to have mixed the two problems.

Why this doesn't work for LIS

You wrote:

PYTHON

```
if nums[i] > nums[j]:
    dp[i][j] = 2 + dp[i + 1][j - 1]
else:
    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1])
```

This recurrence comes from:

TEXT

```
If ends match:
    take both ends

Otherwise:
    discard one end
```

which is exactly the idea behind palindromic subsequences.

For LIS, there is no notion of:

TEXT

```
left end
right end
matching ends
interval [i,j]
```

So an Interval DP state:

PYTHON

```
dp[i][j]
```

doesn't naturally represent the problem.

How to classify LIS

LIS is a **Prefix / Sorted Prefix DP** problem.

State:

PYTHON

```
dp[i]
```

means:

TEXT

```
Length of the longest increasing subsequence
ending at index i
```

Notice:

TEXT

```
ending at i
```

not

TEXT

inside interval [i,j]

Thought Process

Example:

PYTHON

```
nums = [10,9,2,5,3,7,101,18]
```

Suppose we want:

PYTHON

```
dp[5]
```

for value:

PYTHON

```
7
```

Ask:

What previous element can come before 7 in an increasing subsequence?

Look at all previous indices:

TEXT

```
10 9 2 5 3
```

Only:

TEXT

```
2, 5, 3
```

can precede 7.

Therefore:

PYTHON

```
dp[5] = 1 + max(  
    dp[j]  
    for j < 5  
    if nums[j] < nums[5]  
)
```

Recurrence

PYTHON

```
dp[i] = 1
```

(initially)

Then:

PYTHON

```
for j in range(i):
    if nums[j] < nums[i]:
        dp[i] = max(dp[i], dp[j] + 1)
```

Correct LIS Code

PYTHON

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)

        dp = [1] * n

        for i in range(n):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)

        return max(dp)
```

Quick DP Classification Rule

Problem	State
Longest Palindromic Subsequence	dp[i][j]
Longest Palindromic Substring	dp[i][j]
Matrix Chain Multiplication	dp[i][j]
Burst Balloons	dp[i][j]
LIS	dp[i]
House Robber	dp[i]
Decode Ways	dp[i]

If you find yourself thinking:

TEXT

What is the answer for interval [i, j]?

then use Interval DP.

If you're thinking:

TEXT

What is the best answer ending at i?

then use Prefix/LIS-style DP.

USER

For LIS, there is no notion of:

left end right end matching ends interval $[i, j]$

So an Interval DP state:

$dp[i][j]$

doesn't naturally represent the problem. why i mean, if the left is small than right, then we can proceed inside the region, the idea seems right

ASSISTANT

Your intuition is good. The issue is more subtle:

The recurrence you're using doesn't capture what LIS actually asks for.

Let's test your idea.

Suppose:

TEXT

nums = [1, 5, 2, 3]

The LIS is:

TEXT

1, 2, 3

length = 3.

Now consider the interval:

TEXT

[1, 5, 2, 3]
^ ^
i j

Since:

TEXT

1 < 3

your intuition says:

"Great, let's take both ends and solve the inside."

Something like:

TEXT

```
2 + dp[i+1][j-1]
```

But the inside is:

TEXT

```
[5, 2]
```

The LIS inside has length:

TEXT

```
1
```

So you'd get:

TEXT

```
2 + 1 = 3
```

which happens to be correct here.

Let's try another example.

Counterexample

TEXT

```
nums = [1, 100, 2, 3]
```

LIS:

TEXT

```
1, 2, 3
```

length = 3.

Using your interval idea:

TEXT

```
1 < 3
```

SO:

TEXT

```
dp[0][3] = 2 + dp[1][2]
```

Inside:

TEXT

[100, 2]

LIS length = 1.

Therefore:

TEXT

$dp[0][3] = 3$

Still works.

Let's push further.

Failing Example

TEXT

nums = [1, 10, 2, 20]

LIS:

TEXT

1, 2, 20

length = 3.

Your recurrence might say:

TEXT

$1 < 20$

therefore

TEXT

$2 + dp[1][2]$

Inside:

TEXT

[10, 2]

LIS length = 1.

Result:

TEXT

3

Still okay!

This makes the idea look convincing.

Now the real problem:

TEXT

```
nums = [1, 8, 2, 3, 4]
```

LIS:

TEXT

```
1, 2, 3, 4
```

length = 4.

Your recurrence:

TEXT

```
1 < 4
```

so:

TEXT

```
2 + dp[1][3]
```

Inside:

TEXT

```
[8, 2, 3]
```

LIS length = 2

(2, 3)

Therefore:

TEXT

```
2 + 2 = 4
```

Still works!

So what's wrong?

The issue is that **the LIS inside doesn't necessarily fit between the chosen endpoints.**

Consider:

TEXT

```
nums = [5, 1, 2, 3, 4]
```

The LIS is:

TEXT

```
1, 2, 3, 4
```

length = 4.

If you decide to take both ends:

TEXT

5 4

you can't.

Why?

Because:

TEXT

5 < 4

is false.

The left endpoint prevents every increasing sequence inside from being extended.

Even worse:

TEXT

nums = [1, 10, 20, 5]

LIS:

TEXT

1, 10, 20

length = 3.

Suppose you look at:

TEXT

1 5

Since:

TEXT

1 < 5

you might want:

TEXT

2 + dp[1][2]

But:

TEXT

dp[1][2]

is the LIS length of:

TEXT

10, 20

which is 2.

Then you'd get:

TEXT

2 + 2 = 4

which is impossible.

Why?

Because:

TEXT

1, 10, 20, 5

is **not increasing**.

The subsequence inside must also satisfy:

TEXT

1 < every chosen inside element < 5

Your state `dp[i][j]` doesn't store that information.

This is the key insight:

For palindromes, knowing the interval `[i, j]` is enough because the only thing that matters is:

TEXT

`s[i]`
`s[j]`

and the inside interval.

For LIS, when you choose endpoints, you must know:

TEXT

What was the previous value?
What values are allowed next?

The interval alone does not contain enough information.

That's why LIS is usually defined as:

TEXT

`dp[i] = LIS ending at i`

or

TEXT

`dp[i] = LIS starting at i`

because the crucial information is the **last chosen value**, not the interval boundaries.

A good rule:

Interval DP works when the subproblem is completely determined by a range $[i, j]$.

LIS is not. The answer inside $[i, j]$ depends on what values you've already chosen before entering that range, so $[i, j]$ alone is not a sufficient state.

USER

do the same for this problem , show the though process in solving this

ASSISTANT

Let's do **Longest Increasing Subsequence (LIS)** the same way we did for Decode Ways and Longest Palindromic Subsequence.

Problem

Given:

TEXT

`nums = [10, 9, 2, 5, 3, 7, 101, 18]`

Find the length of the longest strictly increasing subsequence.

Answer:

TEXT

`2, 3, 7, 101`

Length:

TEXT

`4`

Step 1: Classify the problem

Ask:

Is this a Prefix DP or Interval DP?

The answer is **Prefix DP**.

Why?

Because when deciding whether to use `nums[i]`, we only care about elements **before** `i`.

For example:

```
TEXT
10 9 2 5 3 7
      ^
      i
```

To build an increasing subsequence ending at `7`, we look at previous positions.

We never ask:

```
TEXT
What is the answer inside [i,j]?
```

So this is not Interval DP.

Step 2: Define the state

The biggest mistake beginners make is defining:

```
PYTHON
dp[i]
```

as

```
TEXT
LIS in first i elements
```

This is hard to work with.

Instead define:

```
PYTHON
dp[i]
```

as

```
TEXT
Length of the longest increasing subsequence
that ENDS at index i
```

This makes the last element fixed.

Example:

```
TEXT
nums = [10,9,2,5]
```

For:

PYTHON

```
dp[3]
```

we are asking:

TEXT

Longest increasing subsequence ending at 5

Possible subsequences:

TEXT

2, 5

Length:

TEXT

2

Therefore:

PYTHON

```
dp[3] = 2
```

Step 3: Think about the last action

This is the same trick as Decode Ways.

Ask:

What was the element immediately before `nums[i]`?

Suppose:

TEXT

```
nums[i] = 7
```

TEXT

```
10 9 2 5 3 7
           ^
```

What can come before 7?

TEXT

```
2
5
3
```

because they are smaller than 7.

Not:

TEXT

10
9

because then the sequence wouldn't be increasing.

Step 4: Write the recurrence

Suppose we choose:

TEXT

2 -> 7

Then:

PYTHON

```
dp[i] = dp[index_of_2] + 1
```

Suppose we choose:

TEXT

5 -> 7

Then:

PYTHON

```
dp[i] = dp[index_of_5] + 1
```

We want the best one.

So:

PYTHON

```
dp[i] = 1 + max(dp[j])
```

for all:

PYTHON

```
j < i  
and  
nums[j] < nums[i]
```

Step 5: Base case

Every element alone forms an increasing subsequence.

Example:

```
TEXT
[100]
```

Length:

```
TEXT
1
```

Therefore:

```
PYTHON
dp[i] = 1
```

initially.

Step 6: Example walkthrough

```
TEXT
nums = [10, 9, 2, 5, 3, 7]
```

Initially:

```
TEXT
dp = [1, 1, 1, 1, 1, 1]
```

i = 0

```
TEXT
10
```

Nothing before it.

```
TEXT
dp[0] = 1
```

i = 1

```
TEXT
9
```

Check:

```
TEXT
10 < 9 ?
```

No.

TEXT

`dp[1] = 1`

i = 2

TEXT

2

No smaller previous element.

TEXT

`dp[2] = 1`

i = 3

TEXT

5

Check:

TEXT

`10 < 5 ? no`

`9 < 5 ? no`

`2 < 5 ? yes`

Therefore:

PYTHON

`dp[3] = dp[2] + 1`

TEXT

`= 2`

Now:

TEXT

`dp = [1, 1, 1, 2, 1, 1]`

i = 4

TEXT

3

Check:

TEXT

2 < 3

So:

PYTHON

dp[4] = dp[2] + 1

TEXT

= 2

Now:

TEXT

dp = [1, 1, 1, 2, 2, 1]

i = 5

TEXT

7

Check all previous:

TEXT

2 < 7

5 < 7

3 < 7

Possible values:

TEXT

dp[2]+1 = 2

dp[3]+1 = 3

dp[4]+1 = 3

Take maximum:

TEXT

3

Now:

TEXT

dp = [1, 1, 1, 2, 2, 3]

Final Answer

The LIS may end anywhere.

So return:

```
PYTHON
max(dp)
```

not:

```
PYTHON
dp[n-1]
```

because the longest subsequence doesn't have to end at the last element.

Code

```
PYTHON
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)

        dp = [1] * n

        for i in range(n):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)

        return max(dp)
```

The key insight

For Prefix DP, always ask:

What does `dp[i]` represent?

For LIS, the magic state is:

```
TEXT
dp[i] = longest increasing subsequence ending at i
```

Once you fix the ending position, the recurrence becomes natural:

```
PYTHON
dp[i] = 1 + max(dp[j])
```

over all previous positions `j` that can legally come before `i`.

```
USER
ok, can you do it defining the subproblem as dp[i] = lis until i
```

ASSISTANT

Yes. You **can** define

PYTHON

```
dp[i] = LIS length using elements from nums[0...i]
```

but you'll discover that the recurrence becomes awkward.

Let's try.

Definition

PYTHON

```
dp[i] = LIS length in the prefix nums[0...i]
```

Example:

TEXT

```
nums = [10, 9, 2, 5]
```

TEXT

```
dp[0] = 1
dp[1] = 1
dp[2] = 1
dp[3] = 2
```

because the LIS in:

TEXT

```
[10, 9, 2, 5]
```

is:

TEXT

```
2, 5
```

of length 2.

Try to derive a recurrence

Suppose we're computing:

PYTHON

```
dp[i]
```

Ask:

How does the LIS in the first i elements relate to the LIS in the first $i-1$ elements?

Two possibilities:

Case 1

The LIS does not use `nums[i]`.

Then:

PYTHON

```
dp[i] = dp[i-1]
```

Case 2

The LIS uses `nums[i]`.

Now we need:

What is the best increasing subsequence that can end before `i` and be extended by `nums[i]` ?

Here's the problem.

Knowing only:

PYTHON

```
dp[i-1]
```

doesn't tell us:

- what value the LIS ends with
- whether that ending value is smaller than `nums[i]`

Counterexample

Consider:

TEXT

```
[1, 100]
```

PYTHON

```
dp[1] = 2
```

Now add:

TEXT

```
2
```

SO:

TEXT

```
[1, 100, 2]
```

The LIS length in the prefix is still:

TEXT

2

But can we extend the LIS represented by `dp[1]` ?

No.

TEXT

1, 100, 2

is not increasing.

The issue is:

PYTHON

```
dp[1] = 2
```

doesn't tell us that the subsequence ends at `100` .

Missing information

To extend a sequence with `nums[i]` , we need to know:

TEXT

What value does the previous subsequence end with?

The state

PYTHON

```
dp[i] = LIS in first i elements
```

doesn't store that.

So it is **not a sufficient state** for an easy recurrence.

Why "ending at i" works

Define:

PYTHON

```
dp[i] = LIS ending at i
```

Now the last element is fixed:

TEXT

```
... -> nums[i]
```

To extend it, we only need to look at:

```
PYTHON
```

```
j < i
```

with

```
PYTHON
```

```
nums[j] < nums[i]
```

giving:

```
PYTHON
```

```
dp[i] = 1 + max(dp[j])
```

which is simple.

Can we still solve it with "LIS until i"?

Yes, but we need another state.

Let:

```
PYTHON
```

```
end[i] = LIS ending at i
```

Then:

```
PYTHON
```

```
dp[i] = LIS in first i elements
```

can be computed as:

```
PYTHON
```

```
dp[i] = max(dp[i-1], end[i])
```

For example:

```
TEXT
```

```
nums = [10, 9, 2, 5, 3, 7]
```

```
TEXT
```

```
end = [1, 1, 1, 2, 2, 3]
```

and

```
TEXT
```

```
dp = [1, 1, 1, 2, 2, 3]
```

In this particular problem, `dp` becomes redundant because:

PYTHON

```
answer = max(end)
```

already gives the LIS.

This illustrates a general DP principle:

A state is good if it contains exactly the information needed to build larger states.

`dp[i] = LIS in first i elements` loses too much information (the ending value), while
`dp[i] = LIS ending at i` keeps exactly the information needed for the recurrence.

USER

This is a prefix dp

```
class Solution: # dp[i] = length of LIS ending at index i ( meaning the last element is nums[i] )
def lengthOfLIS(self, nums: List[int]) -> int:
    n = len(nums)
    dp = [1] * n
    for i in range(1, n):
        dp[i] = max(dp[i], max([dp[j] + 1 if nums[j] < nums[i] else 0 for j in range(i)]))
    return dp[n - 1]
```

ASSISTANT

Your state definition is correct:

PYTHON

```
dp[i] = length of LIS ending at index i
```

But there are **two issues** in the code.

Issue 1: Wrong return value

You wrote:

PYTHON

```
return dp[n - 1]
```

`dp[n-1]` is the LIS **ending at the last element**, not the LIS of the entire array.

Example:

PYTHON

```
nums = [1, 2, 3, 0]
```

DP becomes:

PYTHON

```
[1, 2, 3, 1]
```

Your code returns:

PYTHON

```
1
```

But the answer is:

PYTHON

```
3
```

So you need:

PYTHON

```
return max(dp)
```

Issue 2: Unnecessary list creation

You wrote:

PYTHON

```
max([dp[j] + 1 if nums[j] < nums[i] else 0 for j in range(i)])
```

This creates a whole list.

You can write:

PYTHON

```
max(dp[j] + 1 if nums[j] < nums[i] else 0 for j in range(i))
```

using a generator expression.

Cleaner version

PYTHON

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)

        dp = [1] * n

        for i in range(1, n):
            dp[i] = max(
                dp[i],
                max(
                    (dp[j] + 1 if nums[j] < nums[i] else 0)
                    for j in range(i)
                )
            )

        return max(dp)
```

Most common formulation

Most people write it as:

PYTHON

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)

        dp = [1] * n

        for i in range(n):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)

        return max(dp)
```

This version makes the recurrence very explicit:

$\forall dp[i] = 1 + \max(dp[j])$

for all

$\forall j < i \text{ and } nums[j] < nums[i]$

which directly matches the state definition "LIS ending at **i**".

USER

Failed Test Case Input

nums=[1,3,6,7,9,4,10,5,6] Your Output

5 Expected Output

6 Code | Python

This is a prefix dp

```
class Solution: # dp[i] = length of LIS ending at index i ( meaning the last element is nums[i] )
def lengthOfLIS(self, nums: List[int]) -> int:
    n = len(nums)
    dp = [1] * n
    for i in range(1, n):
        dp[i] = max(dp[i], max(dp[j] + 1 if nums[j] < nums[i] else 0 for j in range(i)))
    return dp[n - 1]
```

ASSISTANT

The problem is still the return statement.

Your state is:

PYTHON

```
dp[i] = length of LIS ending at index i
```

So `dp[n-1]` means:

TEXT

length of LIS ending at the last element

not

TEXT

length of LIS in the entire array

Let's trace your test case:

PYTHON

```
nums = [1,3,6,7,9,4,10,5,6]
```

The DP becomes:

PYTHON

```
dp = [1,2,3,4,5,3,6,4,5]
```

Interpretation:

TEXT

```
dp[0] = 1 -> [1]
dp[1] = 2 -> [1,3]
dp[2] = 3 -> [1,3,6]
dp[3] = 4 -> [1,3,6,7]
dp[4] = 5 -> [1,3,6,7,9]
dp[5] = 3 -> [1,3,4]
dp[6] = 6 -> [1,3,6,7,9,10]
dp[7] = 4 -> [1,3,4,5]
dp[8] = 5 -> [1,3,4,5,6]
```

Notice:

PYTHON

```
dp[8] = 5
```

because the LIS ending at the last element 6 is:

TEXT

1, 3, 4, 5, 6

length 5.

But the overall LIS is:

TEXT

1, 3, 6, 7, 9, 10

length 6,

which ends at index 6, not index 8.

Therefore:

PYTHON

```
return max(dp)
```

not

PYTHON

```
return dp[n-1]
```

Fixed code

PYTHON

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        dp = [1] * n

        for i in range(1, n):
            dp[i] = max(
                dp[i],
                max(
                    (dp[j] + 1 if nums[j] < nums[i] else 0)
                    for j in range(i)
                )
            )

        return max(dp)
```

Or more commonly:

PYTHON

```
class Solution:
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        dp = [1] * n

        for i in range(n):
            for j in range(i):
                if nums[j] < nums[i]:
                    dp[i] = max(dp[i], dp[j] + 1)

        return max(dp)
```

The key distinction is:

- `dp[i]` = LIS **ending at i**
- `max(dp)` = LIS **anywhere in the array**.

USER

Given an integer array `nums`, find a subarray that has the largest product within the array and return it.

A subarray is a contiguous non-empty sequence of elements within an array.

You can assume the output will fit into a 32-bit integer.

Example 1:

Input: `nums = [1,2,-3,4]`

Output: 4

ASSISTANT

Let's derive it from scratch, just like we did for LIS.

Step 1: Classify the problem

We're asked:

TEXT

Maximum product of a contiguous subarray

Keywords:

TEXT

subarray
contiguous

At first glance, it looks like Prefix DP:

PYTHON

`dp[i]`

representing something about subarrays ending at `i`.

Step 2: Try a simple state

Let's define:

PYTHON

`dp[i]` = maximum product of a subarray ending at `i`

Example:

TEXT

`nums = [2,3,-2]`

TEXT

`dp[0] = 2`
`dp[1] = 6`
`dp[2] = ?`

For `-2`:

TEXT

`6 * (-2) = -12`

SO:

TEXT

```
dp[2] = -2
```

This seems fine.

Step 3: Find the problem

Consider:

TEXT

```
nums = [-2, 3, -4]
```

Answer:

TEXT

```
(-2) * 3 * (-4) = 24
```

Let's compute with our state.

TEXT

```
dp[0] = -2  
dp[1] = 3
```

Now at `-4`:

Using only the maximum previous product:

TEXT

```
3 * (-4) = -12
```

or

TEXT

```
-4
```

giving:

TEXT

```
dp[2] = -4
```

But the correct answer should be:

TEXT

```
24
```

We lost information!

Why?

Because multiplying by a negative flips signs.

A very negative number can suddenly become the largest positive number.

Example:

TEXT

```
(-6) * (-4) = 24
```

The previous **minimum** product is just as important as the previous maximum product.

Step 4: Fix the state

Define:

PYTHON

```
maxProd[i]
```

=

TEXT

```
maximum product ending at i
```

and

PYTHON

```
minProd[i]
```

=

TEXT

```
minimum product ending at i
```

Step 5: Think about the last action

For each number:

PYTHON

```
nums[i]
```

there are three possibilities.

Start a new subarray:

PYTHON

```
nums[i]
```

Extend previous maximum:

PYTHON

```
maxProd[i-1] * nums[i]
```

Extend previous minimum:

PYTHON

```
minProd[i-1] * nums[i]
```

because a negative $\times$ negative may become the largest positive.

Therefore:

PYTHON

```
maxProd[i] = max(
    nums[i],
    maxProd[i-1] * nums[i],
    minProd[i-1] * nums[i]
)
```

and

PYTHON

```
minProd[i] = min(
    nums[i],
    maxProd[i-1] * nums[i],
    minProd[i-1] * nums[i]
)
```

Example

TEXT

```
nums = [-2, 3, -4]
```

Initially:

TEXT

```
maxProd = -2
minProd = -2
ans = -2
```

i = 1

TEXT

```
num = 3
```

Candidates:

TEXT

```
3
-2 * 3 = -6
-2 * 3 = -6
```

TEXT

```
maxProd = 3  
minProd = -6
```

i = 2

TEXT

```
num = -4
```

Candidates:

TEXT

```
-4  
3 * (-4) = -12  
-6 * (-4) = 24
```

TEXT

```
maxProd = 24  
minProd = -12
```

Answer:

TEXT

```
24
```

Correct.

Space Optimization

Notice we only need the previous values.

Instead of arrays:

PYTHON

```
maxProd  
minProd
```

can be variables.

Code

PYTHON

```
class Solution:  
    def maxProduct(self, nums: List[int]) -> int:  
  
        cur_max = nums[0]  
        cur_min = nums[0]  
  
        ans = nums[0]
```

```

for i in range(1, len(nums)):

    num = nums[i]

    candidates = (
        num,
        cur_max * num,
        cur_min * num
    )

    cur_max = max(candidates)
    cur_min = min(candidates)

    ans = max(ans, cur_max)

return ans

```

DP Classification

This is a **Prefix DP**.

The state is:

TEXT

```

max product ending at i
min product ending at i

```

The key observation is:

For sums, only the previous maximum matters.

For products, both the previous maximum and previous minimum matter because multiplication by a negative number can swap them.

USER

```

class Solution: def maxProduct(self, nums: List[int]) -> int: n = len(nums) maxProd = [0] n minProd = [0] n
minProd[0] = nums[0] maxProd[0] = nums[0] for i in range(1, n):

```

Traceback (most recent call last): File "/box/main.py", line 29, in main raise TypeError(f"Your output was {output}, but the expected return type is int") TypeError: Your output was None, but the expected return type is int

Input:

nums=[1,2,-3,4] stdout:

```

[1, 2, -3, 4] candidates = ( nums[i], nums[i] maxProd[i - 1], nums[i] minProd[i - 1] )

```

```

maxProd[i] = max(candidates) minProd[i] = min(candidates) print(maxProd)

```

ASSISTANT

The error message tells you exactly what's wrong:

TEXT

TypeError: Your output was None, but the expected return type is int

Your function ends with:

PYTHON

```
print(maxProd)
```

but it never does:

PYTHON

```
return ...
```

So Python returns:

PYTHON

```
None
```

by default.

Currently your code computes:

PYTHON

```
maxProd = [1, 2, -3, 4]
```

for:

PYTHON

```
nums = [1,2,-3,4]
```

and then exits.

You need to return the maximum product found.

For your DP arrays:

PYTHON

```
class Solution:
    def maxProduct(self, nums: List[int]) -> int:
        n = len(nums)

        maxProd = [0] * n
        minProd = [0] * n

        maxProd[0] = nums[0]
        minProd[0] = nums[0]

        for i in range(1, n):

            candidates = (
                nums[i],
                nums[i] * maxProd[i - 1],
                nums[i] * minProd[i - 1]
            )
```

```
        maxProd[i] = max(candidates)
        minProd[i] = min(candidates)

    return max(maxProd)
```

Let's verify with:

PYTHON

```
nums = [1,2,-3,4]
```

DP becomes:

PYTHON

```
maxProd = [1, 2, -3, 4]
minProd = [1, 2, -6, -24]
```

Answer:

PYTHON

```
max(maxProd) = 4
```

which matches the expected output.

A small optimization is to keep track of the answer while filling the DP:

PYTHON

```
ans = nums[0]

for i in range(1, n):
    ...
    ans = max(ans, maxProd[i])

return ans
```

This avoids scanning `maxProd` again at the end.

USER

Failed Test Case Input

nums=[2,3,-2,4] Your Output

4 Expected Output

6

ASSISTANT

Your recurrence is correct. The problem is in the assumption:

```
PYTHON
```

```
return max(maxProd)
```

Let's inspect the DP for:

```
PYTHON
```

```
nums = [2, 3, -2, 4]
```

i = 0

```
PYTHON
```

```
maxProd[0] = 2
```

```
minProd[0] = 2
```

i = 1 (3)

Candidates:

```
PYTHON
```

```
3
```

```
2 * 3 = 6
```

```
2 * 3 = 6
```

```
PYTHON
```

```
maxProd[1] = 6
```

```
minProd[1] = 3
```

i = 2 (-2)

Candidates:

```
PYTHON
```

```
-2
```

```
6 * (-2) = -12
```

```
3 * (-2) = -6
```

```
PYTHON
```

```
maxProd[2] = -2
```

```
minProd[2] = -12
```

i = 3 (4)

Candidates:

```
PYTHON
```

```
4
```

```
-2 * 4 = -8
```

```
-12 * 4 = -48
```

PYTHON

```
maxProd[3] = 4  
minProd[3] = -48
```

So:

PYTHON

```
maxProd = [2, 6, -2, 4]
```

and:

PYTHON

```
max(maxProd) = 6
```

which is correct.

Therefore, if you're getting 4, your actual code is likely returning:

PYTHON

```
maxProd[n - 1]
```

or

PYTHON

```
maxProd[-1]
```

instead of:

PYTHON

```
max(maxProd)
```

Can you paste the exact current code?

Also, there is a cleaner way to write it:

PYTHON

```
class Solution:  
    def maxProduct(self, nums: List[int]) -> int:  
        n = len(nums)  
  
        maxProd = [0] * n  
        minProd = [0] * n  
  
        maxProd[0] = nums[0]  
        minProd[0] = nums[0]  
  
        ans = nums[0]  
  
        for i in range(1, n):  
            candidates = (  
                nums[i],  
                nums[i] * maxProd[i - 1],  
                nums[i] * minProd[i - 1]  
            )
```

```
        maxProd[i] = max(candidates)
        minProd[i] = min(candidates)

    ans = max(ans, maxProd[i])

    return ans
```

This returns 6 for:

```
PYTHON
[2, 3, -2, 4]
```

and 4 for:

```
PYTHON
[1, 2, -3, 4]
```

which matches the expected outputs.

USER

```
class Solution: def longestCommonSubsequence(self, text1: str, text2: str) -> int: m = len(text1) n = len(text2) dp =
[[0] * n for _ in range(m)] for i in range(n): for j in range(n): if text1[i] == text2[j]: dp[i][j] = 1 + (dp[i - 1][j - 1] if ((i - 1) >=
0 and (j - 1) >= 0) else 0) else: dp[i][j] = max(dp[i][j] - 1 if (j - 1 >= 0) else 0 , dp[i - 1][j - 1] if (i - 1 >= 0 and j - 1 >= 0)
else 0) return dp[n - 1][n - 1]
```

ASSISTANT

Your code has a few issues. Let's derive the correct DP and then point out the mistakes.

State

For LCS, define:

```
PYTHON
dp[i][j]
```

as:

```
TEXT
Length of the LCS between:
text1[0..i]
text2[0..j]
```

This is a **Two Sequences DP** problem.

Recurrence

If characters match

```
PYTHON
text1[i] == text2[j]
```

Then we use both characters:

```
PYTHON
dp[i][j] = 1 + dp[i-1][j-1]
```

If characters don't match

Suppose:

```
TEXT
text1 = abc
text2 = adc
```

At some `(i, j)`, the current characters differ.

We have two choices:

1. Skip `text1[i]`

```
PYTHON
dp[i-1][j]
```

2. Skip `text2[j]`

```
PYTHON
dp[i][j-1]
```

Take the best:

```
PYTHON
dp[i][j] = max(
    dp[i-1][j],
    dp[i][j-1]
)
```

Notice:

'LNot:

```
PYTHON
dp[i-1][j-1]
```

because that skips **both** characters.

Mistakes in your code

1. Wrong loop

You wrote:

PYTHON

```
for i in range(n):  
    for j in range(n):
```

But:

PYTHON

```
m = len(text1)  
n = len(text2)
```

So it should be:

PYTHON

```
for i in range(m):  
    for j in range(n):
```

Otherwise you'll get index errors when `m != n`.

2. Wrong recurrence for mismatch

You wrote:

PYTHON

```
dp[i][j] = max(  
    dp[i][j-1],  
    dp[i-1][j-1]  
)
```

But it should be:

PYTHON

```
dp[i][j] = max(  
    dp[i-1][j],  
    dp[i][j-1]  
)
```

3. Wrong return

You wrote:

PYTHON

```
return dp[n-1][n-1]
```

But the table size is:

PYTHON

```
m x n
```

So it should be:

PYTHON

```
return dp[m-1][n-1]
```

Easier Version (1-based indexing)

Most people avoid boundary checks by using an extra row and column.

PYTHON

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m = len(text1)
        n = len(text2)

        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if text1[i - 1] == text2[j - 1]:
                    dp[i][j] = 1 + dp[i - 1][j - 1]
                else:
                    dp[i][j] = max(
                        dp[i - 1][j],
                        dp[i][j - 1]
                    )

        return dp[m][n]
```

Thought Process

For Two-Sequences DP, always ask:

What happens to the last characters of the two prefixes?

For:

PYTHON

```
dp[i][j]
```

there are only two cases:

Match

TEXT

```
...A
...A
```

Take them:

PYTHON

```
1 + dp[i-1][j-1]
```

No Match

TEXT

...A
...B

One of them cannot be in the common subsequence.

So try:

PYTHON

```
dp[i-1][j]
```

or

PYTHON

```
dp[i][j-1]
```

and take the maximum.

That's the standard pattern for LCS and many related two-sequence DP problems.

USER

what type of problem is this ???

ASSISTANT

Longest Common Subsequence (LCS) belongs to a very important DP category:

Two-Sequence DP

The state depends on **two indices**, one in each sequence.

Typical state:

PYTHON

```
dp[i][j]
```

means something about:

TEXT

```
text1[0...i]  
text2[0...j]
```

or

TEXT

```
first i characters of text1  
first j characters of text2
```

How to recognize it

Whenever you see:

TEXT

two strings
two arrays
compare sequences
alignment
matching
common
edit
transform one string into another

think:

TEXT

Two-Sequence DP

The thought process

For LCS, define:

PYTHON

```
dp[i][j]
```

as:

TEXT

LCS length between
text1[0...i]
and
text2[0...j]

Now ask:

What happens with the last characters?

Case 1: Match

TEXT

abcD
xyzD

Take both:

PYTHON

```
dp[i][j] = 1 + dp[i-1][j-1]
```

Case 2: Don't match

TEXT

abcd
xyzE

One of them must be discarded.

Either:

PYTHON

```
dp[i-1][j]
```

or

PYTHON

```
dp[i][j-1]
```

Take the better one.

DP Classification Cheat Sheet

1. Prefix DP

State:

PYTHON

```
dp[i]
```

Examples:

- House Robber
- Decode Ways
- LIS
- Climbing Stairs
- Maximum Product Subarray

Question usually looks like:

TEXT

What is the best answer up to index i?

2. Interval DP

State:

PYTHON

```
dp[i][j]
```

where **i** and **j** are in the **same array/string**.

Examples:

- Longest Palindromic Subsequence

- Longest Palindromic Substring
- Matrix Chain Multiplication
- Burst Balloons

Question usually looks like:

TEXT

What is the answer inside interval $[i, j]$?

3. Two-Sequence DP

State:

PYTHON

`dp[i][j]`

but now:

PYTHON

`i -> sequence 1`
`j -> sequence 2`

Examples:

- Longest Common Subsequence
- Edit Distance
- Distinct Subsequences
- Wildcard Matching
- Interleaving String

Question usually looks like:

TEXT

What is the answer for the first i chars of `string1` and first j chars of `string2`?

4. Grid DP

State:

PYTHON

`dp[r][c]`

Examples:

- Unique Paths
- Minimum Path Sum
- Cherry Pickup

Question usually looks like:

TEXT

What is the best answer to reach cell (r,c) ?

5. Knapsack DP

State:

PYTHON

```
dp[i][w]
```

Examples:

- 0/1 Knapsack
- Partition Equal Subset Sum
- Target Sum

Question usually looks like:

TEXT

Can I make sum w using first i items?

For interviews, a useful first question is:

"What do the dimensions of my DP state represent?"

- One index !' usually Prefix DP.
- Two indices on the **same string/array** !' often Interval DP.
- Two indices on **different strings/arrays** !' Two-Sequence DP.
- Row and column !' Grid DP.
- Item and capacity/sum !' Knapsack DP.

LCS is the classic **Two-Sequence DP** problem.

USER

solve word break problem

ASSISTANT

Let's solve **Word Break** using the same thought process.

Problem

Given:

PYTHON

```
s = "leetcode"  
wordDict = ["leet", "code"]
```

Can we split the string into valid dictionary words?

TEXT

leet + code

Answer:

PYTHON

True

Step 1: Classify the problem

Ask:

What is the answer for the prefix of the string?

This immediately suggests:

TEXT

Prefix DP

because we're building the answer from left to right.

Step 2: Define the state

Let:

PYTHON

dp[i]

mean:

TEXT

Can the prefix `s[0:i]` be segmented into dictionary words?

Notice:

PYTHON

`s[0:i]`

is Python slicing.

So:

PYTHON

```
dp[0]
```

means:

TEXT

Can the empty string be segmented?

Answer:

PYTHON

```
True
```

because we've successfully segmented nothing.

Step 3: Find the last action

Suppose:

PYTHON

```
s = "leetcode"
```

and we're trying to compute:

PYTHON

```
dp[8]
```

for:

TEXT

```
"leetcode"
```

Ask:

What could be the last word used?

Maybe:

TEXT

```
"code"
```

Then before that we must have successfully segmented:

TEXT

```
"leet"
```

So:

```
PYTHON
```

```
dp[8] = dp[4]
```

provided:

```
PYTHON
```

```
s[4:8] == "code"
```

Generalize

Suppose the last word starts at `j`.

Then:

```
TEXT
```

```
s[0:j] | s[j:i]  
previous    last word
```

For `dp[i]` to be True:

1. Prefix before it must be valid:

```
PYTHON
```

```
dp[j]
```

2. Last piece must be a dictionary word:

```
PYTHON
```

```
s[j:i] in wordDict
```

Therefore:

```
PYTHON
```

```
dp[i] = True
```

if there exists some `j` such that:

```
PYTHON
```

```
dp[j] and s[j:i] in wordDict
```

Recurrence

```
PYTHON
```

```
for j in range(i):  
    if dp[j] and s[j:i] in words:  
        dp[i] = True
```

Example

PYTHON

```
s = "leetcode"  
wordDict = ["leet", "code"]
```

Initialize:

PYTHON

```
dp = [True, False, False, False, False, False, False, False, False]
```

i = 4

Check:

PYTHON

```
s[0:4] = "leet"
```

Dictionary contains it.

PYTHON

```
dp[4] = dp[0] = True
```

i = 8

Check:

PYTHON

```
s[4:8] = "code"
```

Dictionary contains it.

PYTHON

```
dp[8] = dp[4] = True
```

Answer:

PYTHON

```
dp[8]
```

is:

PYTHON

```
True
```

Code

PYTHON

```
class Solution:
    def wordBreak(self, s: str, wordDict: List[str]) -> bool:

        words = set(wordDict)

        n = len(s)

        dp = [False] * (n + 1)

        dp[0] = True

        for i in range(1, n + 1):

            for j in range(i):

                if dp[j] and s[j:i] in words:
                    dp[i] = True
                    break

        return dp[n]
```

Why this is Prefix DP

State:

PYTHON

dp[i]

depends on:

PYTHON

```
dp[0]
dp[1]
...
dp[i-1]
```

which are all smaller prefixes.

So the pattern is:

TEXT

Prefix DP

just like:

- Decode Ways
- Climbing Stairs
- LIS
- House Robber

Recognition Pattern

Whenever you see:

TEXT

Can a string be split?
Can a prefix be formed?
Can we reach position i?
Can we segment?

think:

PYTHON

```
dp[i]
```

where `i` represents a prefix boundary.

For Word Break specifically:

PYTHON

```
dp[i] = can s[0:i] be formed?
```

is the natural state.

USER

You are given an array of positive integers `nums`.

Return true if you can partition the array into two subsets, `subset1` and `subset2` where `sum(subset1) == sum(subset2)`. Otherwise, return false.

Example 1:

Input: `nums = [1,2,3,4]`

Output: true Explanation: The array can be partitioned as `[1, 4]` and `[2, 3]`.

ASSISTANT

Let's derive it from scratch.

Step 1: Understand the problem

We want:

TEXT

```
sum(subset1) = sum(subset2)
```

Suppose total sum is:

PYTHON

```
total = sum(nums)
```

Then:

TEXT

$\text{subset1} + \text{subset2} = \text{total}$

and

TEXT

$\text{subset1} = \text{subset2}$

Therefore:

TEXT

$2 * \text{subset1} = \text{total}$

So:

TEXT

$\text{subset1} = \text{total} / 2$

The problem becomes:

Can we find a subset whose sum is exactly $\text{total} // 2$?

Example

PYTHON

```
nums = [1,2,3,4]
```

Total:

TEXT

10

Target:

TEXT

5

Can we form sum 5?

TEXT

$1 + 4 = 5$

Yes.

Answer:

TEXT

True

Step 2: Classify the problem

Ask:

For each number, do I take it or not take it?

That's the hallmark of:

TEXT

Knapsack DP

More specifically:

TEXT

Subset Sum DP

Step 3: Define the state

Let:

PYTHON

```
dp[i][s]
```

mean:

TEXT

Using the first i numbers,
can we form sum s ?

Boolean DP.

Step 4: Think about the last action

Suppose we're considering:

PYTHON

```
nums[i-1]
```

(we use $i-1$ because DP is usually 1-indexed)

For sum s , we have two choices.

Don't take the number

Then:

```
PYTHON
```

```
dp[i][s] = dp[i-1][s]
```

Take the number

Suppose current number is:

```
PYTHON
```

```
num = nums[i-1]
```

Then we must have already formed:

```
PYTHON
```

```
s - num
```

using previous elements.

So:

```
PYTHON
```

```
dp[i][s] = dp[i-1][s-num]
```

if:

```
PYTHON
```

```
s >= num
```

Combine them

```
PYTHON
```

```
dp[i][s] =  
dp[i-1][s]  
or  
dp[i-1][s-num]
```

Step 5: Base case

Sum 0 is always possible.

Take nothing.

```
PYTHON
```

```
dp[i][0] = True
```

for every `i`.

Example

PYTHON

```
nums = [1,2,3,4]
```

Target:

PYTHON

```
5
```

Eventually we'll discover:

TEXT

```
1 + 4 = 5
```

SO:

PYTHON

```
dp[n][5] = True
```

2D DP Code

PYTHON

```
class Solution:
    def canPartition(self, nums: List[int]) -> bool:

        total = sum(nums)

        if total % 2:
            return False

        target = total // 2

        n = len(nums)

        dp = [[False] * (target + 1) for _ in range(n + 1)]

        for i in range(n + 1):
            dp[i][0] = True

        for i in range(1, n + 1):
            num = nums[i - 1]

            for s in range(1, target + 1):
                dp[i][s] = dp[i - 1][s]

                if s >= num:
                    dp[i][s] |= dp[i - 1][s - num]

        return dp[n][target]
```

Space Optimization

Notice:

PYTHON

```
dp[i][*]
```

depends only on:

PYTHON

```
dp[i-1][*]
```

So we can compress to 1D.

State:

PYTHON

```
dp[s]
```

means:

TEXT

Can we form sum s?

Important Trick

Iterate backwards:

PYTHON

```
for s in range(target, num - 1, -1):
```

Otherwise we'd reuse the same number multiple times.

Optimized Code

PYTHON

```
class Solution:
    def canPartition(self, nums: List[int]) -> bool:

        total = sum(nums)

        if total % 2:
            return False

        target = total // 2

        dp = [False] * (target + 1)

        dp[0] = True

        for num in nums:

            for s in range(target, num - 1, -1):
                dp[s] |= dp[s - num]

        return dp[target]
```

Recognition Pattern

Whenever you see:

TEXT

Choose or don't choose
Subset
Target sum
Partition
Can we form sum X?

Think:

TEXT

Knapsack / Subset Sum DP

Typical state:

PYTHON

```
dp[i][sum]
```

or the optimized version:

PYTHON

```
dp[sum]
```

This problem is one of the classic **0/1 Knapsack (Subset Sum)** problems.

USER

why did you define the subproblem to be like this

ASSISTANT

This is actually the most important DP skill:

Choosing the right state.

Let's see why I chose

PYTHON

```
dp[i][s]
```

to mean:

TEXT

Using the first i numbers,
can we form sum s ?

instead of something else.

Start from the question

The original problem asks:

TEXT

Can we find a subset whose sum is target?

where

PYTHON

```
target = total // 2
```

So the two pieces of information that seem relevant are:

1. How many numbers have we considered?
2. What sum are we trying to make?

That immediately suggests:

PYTHON

```
dp[i][s]
```

Think about the last decision

Suppose we're looking at:

PYTHON

```
dp[i][s]
```

and the current number is:

PYTHON

```
nums[i-1]
```

Ask:

Did I use this number or not?

There are only two possibilities.

Not used

Then the problem becomes:

PYTHON

```
dp[i-1][s]
```

Used

Then I must have already formed:

```
PYTHON
```

```
s = nums[i-1]
```

using the previous numbers.

So:

```
PYTHON
```

```
dp[i-1][s - nums[i-1]]
```

Notice how naturally the recurrence appears.

That's usually a sign you've chosen a good state.

Why not define

```
PYTHON
```

```
dp[i]
```

?

Maybe:

```
PYTHON
```

```
dp[i] = can I partition first i numbers?
```

Let's try.

Example:

```
PYTHON
```

```
nums = [1,2,3,4]
```

Suppose:

```
PYTHON
```

```
dp[3]
```

is True.

What does that tell me?

```
TEXT
```

```
Some subset sum is achievable.
```

But:

TEXT

Which sum?

We don't know.

To process the next number, we need to know which sums are already possible.

So:

PYTHON

```
dp[i]
```

doesn't contain enough information.

Why not define

PYTHON

```
dp[s]
```

directly?

Actually, we can!

That's the optimized version.

PYTHON

```
dp[s]
```

means:

TEXT

Can I form sum s
using the numbers processed so far?

But this is easier to discover after understanding:

PYTHON

```
dp[i][s]
```

first.

General DP state selection trick

Whenever you're stuck, ask:

What information do I need to make the next decision?

For this problem, to decide whether I can make sum s , I need:

TEXT

Which numbers are available?
What sum am I trying to form?

That gives:

PYTHON

```
dp[i][s]
```

Compare with LIS

For LIS, I chose:

PYTHON

```
dp[i]
```

because the important information was:

TEXT

What is the best increasing subsequence ending at i?

We didn't care about sums.

Compare with LCS

For LCS, I chose:

PYTHON

```
dp[i][j]
```

because the important information was:

TEXT

How much of string1 have I used?
How much of string2 have I used?

Two sequences !' two indices.

Compare with Partition Equal Subset Sum

The important information is:

TEXT

How many items can I use?
What sum am I trying to build?

Therefore:

PYTHON

```
dp[i][s]
```

is the natural state.

A useful rule for knapsack-style problems is:

If the problem says "choose elements to reach a target", the state is very often:

```
``python
dp[i][target]
meaning "using the first i items, can I achieve this target?" or "what is the best value for this target?" depending
on the problem.
```

USER

what type is this

ASSISTANT

If you're referring to **Partition Equal Subset Sum**, then its DP category is:

Knapsack DP (0/1 Knapsack)

More specifically:

TEXT

Subset Sum DP

because each number can be:

TEXT

Take it once
or
Don't take it

Recognition Pattern

When a problem contains phrases like:

TEXT

Subset
Partition
Target Sum
Can we form X?
Choose elements

think:

TEXT

Knapsack

Why it's Knapsack

For each element:

PYTHON

```
num = nums[i]
```

you have exactly two choices:

TEXT

Take num
Don't take num

and you're trying to achieve:

TEXT

```
target = total_sum // 2
```

This is the classic 0/1 Knapsack decision process.

DP State Pattern

Most knapsack problems use:

PYTHON

```
dp[i][s]
```

where:

TEXT

i = items considered
s = target/capacity/sum

Examples:

Problem	State
Partition Equal Subset Sum	dp[i][sum]
Subset Sum	dp[i][sum]
Target Sum	dp[i][sum]
0/1 Knapsack	dp[i][capacity]
Coin Change	dp[i][amount] or dp[amount]

DP Classification Cheat Sheet

Prefix DP

PYTHON

```
dp[i]
```

Examples:

- LIS
- Decode Ways
- House Robber
- Maximum Product Subarray

Question:

TEXT

Best answer up to/ending at i?

Interval DP

PYTHON

```
dp[i][j]
```

Examples:

- Longest Palindromic Subsequence
- Longest Palindromic Substring
- Burst Balloons

Question:

TEXT

Answer inside interval [i,j]?

Two-Sequence DP

PYTHON

```
dp[i][j]
```

Examples:

- LCS
- Edit Distance
- Distinct Subsequences

Question:

TEXT

Answer using first i chars of string1
and first j chars of string2?

Grid DP

PYTHON

```
dp[r][c]
```

Examples:

- Unique Paths
- Minimum Path Sum

Question:

TEXT

Best way to reach cell (r,c)?

Knapsack DP

PYTHON

```
dp[i][target]
```

Examples:

- Partition Equal Subset Sum
- Subset Sum
- Target Sum
- 0/1 Knapsack

Question:

TEXT

Using first i items,
can I achieve target?

So **Partition Equal Subset Sum** is a classic **Knapsack / Subset Sum DP** problem.

USER

or s in range(target, num - 1, -1): why this range ?//

ASSISTANT

This is one of the most important details in **0/1 Knapsack**.

The loop:

PYTHON

```
for s in range(target, num - 1, -1):
```

means:

PYTHON

```
s = target, target-1, ..., num
```

Why not go forward?

Suppose:

PYTHON

```
nums = [2]  
target = 4
```

Initially:

PYTHON

```
dp = [True, False, False, False, False]
```

meaning:

TEXT

```
sum 0 is possible
```

Forward iteration

PYTHON

```
for s in range(num, target + 1):
```

which is:

PYTHON

```
for s in range(2, 5):
```

s = 2

PYTHON

```
dp[2] |= dp[0]
```

becomes:

PYTHON

```
dp[2] = True
```

Current DP:

TEXT

```
[True, False, True, False, False]
```

s = 4

Now:

```
PYTHON
dp[4] |= dp[2]
```

But notice:

```
PYTHON
dp[2]
```

was just updated in this same iteration!

So:

```
PYTHON
dp[4] = True
```

Current DP:

```
TEXT
[True, False, True, False, True]
```

This claims:

```
TEXT
4 = 2 + 2
```

But we only had **one** 2!

We accidentally used the same number twice.

What does backward iteration do?

Now do:

```
PYTHON
for s in range(target, num - 1, -1):
```

which is:

```
PYTHON
s = 4, 3, 2
```

s = 4

Check:

```
PYTHON
dp[4] |= dp[2]
```

At this moment:

```
PYTHON
dp[2] == False
```

because we haven't updated it yet.

So:

```
PYTHON
dp[4] = False
```

s = 2

Now:

```
PYTHON
dp[2] |= dp[0]
```

becomes:

```
PYTHON
dp[2] = True
```

Final:

```
TEXT
[True, False, True, False, False]
```

Correct.

Intuition

When processing a number:

```
PYTHON
num
```

you want every update to use the DP values from the **previous row** of the 2D table.

Backward iteration guarantees that:

```
PYTHON
dp[s - num]
```

has not been modified yet during the current number's processing.

So each number is used:

TEXT

at most once

which is exactly the **0/1 Knapsack** rule.

Compare with Coin Change

For Coin Change (unlimited coins), we actually want to reuse a coin multiple times.

Then we iterate:

PYTHON

```
for s in range(num, target + 1):
```

forward.

Example:

TEXT

coin = 2

After making:

TEXT

2

we immediately allow:

TEXT

```
4 = 2 + 2
6 = 2 + 2 + 2
...
```

because the coin can be reused.

Rule to remember

0/1 Knapsack (use item once):

PYTHON

```
for s in range(target, num - 1, -1):
```

+ pbbackward

Unbounded Knapsack / Coin Change (reuse item):

PYTHON

```
for s in range(num, target + 1):
```

'ip forward

The direction of the loop determines whether the current item can be reused within the same iteration.

USER

wait num - 1 - 1 is num - 2 then how the range is till num ???

ASSISTANT

Good catch. The confusion comes from how Python's `range()` works.

When you write:

PYTHON

```
range(target, num - 1, -1)
```

the **third argument** is the step:

PYTHON

```
-1
```

The second argument (`num - 1`) is the **stop value**, and Python **does not include the stop value**.

For example:

PYTHON

```
list(range(10, 4, -1))
```

gives:

PYTHON

```
[10, 9, 8, 7, 6, 5]
```

Notice:

TEXT

4 is NOT included

because `4` is the stop value.

Now suppose:

PYTHON

```
num = 5
```

Then:

PYTHON

```
range(target, num - 1, -1)
```

becomes:

PYTHON

```
range(target, 4, -1)
```

which generates:

TEXT

```
target, ..., 7, 6, 5
```

and stops **before** reaching 4.

So 5 is included.

Let's verify:

PYTHON

```
list(range(10, 5 - 1, -1))
```

is:

PYTHON

```
list(range(10, 4, -1))
```

result:

PYTHON

```
[10, 9, 8, 7, 6, 5]
```

A useful mnemonic:

Forward range

PYTHON

```
range(a, b)
```

produces:

TEXT

```
a, a+1, ..., b-1
```

Backward range

PYTHON

```
range(a, b, -1)
```

produces:

TEXT

```
a, a-1, ..., b+1
```

So the stop value `b` is still excluded.

That's why:

PYTHON

```
for s in range(target, num - 1, -1):
```

iterates over:

TEXT

```
target, target-1, ..., num
```

exactly what we want.

USER

explain the pattern , and why is it useful in finding them

ASSISTANT

You're asking something deeper:

How do I recognize DP patterns, and why are these patterns useful?

This is exactly how experienced programmers approach DP problems.

Why patterns matter

When you see a new DP problem, don't think:

TEXT

```
How do I solve this exact problem?
```

Think:

TEXT

```
What information uniquely describes a subproblem?
```

The answer usually falls into a few recurring patterns.

1. Prefix DP

State:

PYTHON

```
dp[i]
```

means something about:

TEXT

first i elements
or
ending at i

Recognition

Questions involving:

TEXT

array
string
left to right decisions

Examples:

- Climbing Stairs
- House Robber
- Decode Ways
- LIS
- Maximum Product Subarray
- Word Break

Thought process

Ask:

If I know answers for smaller prefixes, can I build the answer for the current position?

Example:

TEXT

Decode Ways

At index i , the last move is:

TEXT

take 1 digit
or
take 2 digits

So:

PYTHON

`dp[i]`

depends on:

PYTHON

`dp[i-1]`
`dp[i-2]`

2. Interval DP

State:

```
PYTHON
dp[i][j]
```

means:

```
TEXT
answer inside interval [i,j]
```

Recognition

Whenever the problem asks about:

```
TEXT
substring
subarray
segment
range
interval
```

Examples:

- Longest Palindromic Subsequence
- Longest Palindromic Substring
- Burst Balloons
- Matrix Chain Multiplication

Thought process

Ask:

What happens to the ends?

For LPS:

```
TEXT
a . . . . a
```

If ends match:

```
PYTHON
2 + dp[i+1][j-1]
```

Otherwise:

PYTHON

```
max(  
    dp[i+1][j],  
    dp[i][j-1]  
)
```

3. Two-Sequence DP

State:

PYTHON

```
dp[i][j]
```

but now:

TEXT

```
i -> first sequence  
j -> second sequence
```

Recognition

Problems containing:

TEXT

```
two strings  
two arrays  
compare  
transform  
match  
common
```

Examples:

- LCS
- Edit Distance
- Distinct Subsequences
- Wildcard Matching

Thought process

Ask:

What happens to the last character of each sequence?

For LCS:

Match:

PYTHON

```
1 + dp[i-1][j-1]
```

No match:

PYTHON

```
max(  
    dp[i-1][j],  
    dp[i][j-1]  
)
```

4. Knapsack DP

State:

PYTHON

```
dp[i][target]
```

or

PYTHON

```
dp[target]
```

Recognition

Keywords:

TEXT

```
subset  
partition  
target sum  
capacity  
weight  
choose items
```

Examples:

- Partition Equal Subset Sum
- Target Sum
- 0/1 Knapsack
- Coin Change

Thought process

Ask:

Do I take this item or not?

For every item:

TEXT

```
Take  
Don't Take
```

Example:

PYTHON

```
dp[i][s]
```

means:

TEXT

Using first i items,
can I make sum s ?

5. Grid DP

State:

PYTHON

```
dp[r][c]
```

Recognition

Problems involving:

TEXT

grid
matrix
board
paths

Examples:

- Unique Paths
- Minimum Path Sum

Thought process

Ask:

From which neighboring cells could I have arrived here?

Example:

PYTHON

```
dp[r][c]
```

depends on:

PYTHON

```
dp[r-1][c]  
dp[r][c-1]
```

The real secret

Don't memorize problem names.

Memorize the **subproblem shape**.

When you read a problem, ask:

Question 1

Is there one sequence?

TEXT

Yes

Think:

TEXT

Prefix DP
or
Interval DP

Question 2

Am I looking at a range?

TEXT

substring
subarray
interval

Then:

TEXT

Interval DP

Question 3

Are there two sequences?

TEXT

two strings
two arrays

Then:

TEXT

Two-Sequence DP

Question 4

Am I choosing items to reach a target?

TEXT

subset
sum
capacity
partition

Then:

TEXT

Knapsack DP

Question 5

Is there a grid?

TEXT

matrix
board
path

Then:

TEXT

Grid DP

Why this helps

Without patterns:

TEXT

500 DP problems
=
500 different solutions

With patterns:

TEXT

500 DP problems
=
5-10 recurring state designs

Once you identify the pattern, the recurrence often becomes much easier to derive because you already know what information the state should contain.

USER

ok , now moving on to greedy problems , how to solve them , any patterns like this ??

ASSISTANT

Yes. Greedy problems have patterns too, but they're harder than DP because:

TEXT

DP:

Try all possibilities and remember answers.

Greedy:

Make a local choice and somehow prove it's globally optimal.

The hardest part of Greedy is usually:

TEXT

How do I know my local choice is safe?

Step 1: How to recognize Greedy

Look for phrases like:

TEXT

minimum number of ...
maximum number of ...
earliest ...
latest ...
smallest ...
largest ...

Often accompanied by:

TEXT

sort
intervals
deadlines
scheduling
resources

Pattern 1: Interval Scheduling

Recognition

You have intervals:

TEXT

[start, end]

and want:

TEXT

maximum non-overlapping intervals
minimum removals
meeting rooms

Examples:

- Non-overlapping Intervals
 - Meeting Rooms
 - Activity Selection
-

Key Greedy

Sort by:

```
TEXT
ending time
```

Why?

The interval that finishes earliest leaves the most room for future intervals.

Example:

```
TEXT
[1,10]
[2,3]
[4,5]
```

Picking:

```
TEXT
[1,10]
```

kills everything.

Picking:

```
TEXT
[2,3]
```

allows more choices later.

Pattern 2: Interval Merging

Recognition:

```
TEXT
merge intervals
insert interval
overlapping intervals
```

Sort by:

```
TEXT
start time
```

Then scan.

Pattern 3: Take Largest / Smallest First

Recognition:

TEXT

```
k largest
k smallest
maximize profit
minimize cost
```

Usually:

TEXT

```
heap
priority queue
```

Examples:

- IPO
- Task Scheduler
- K Closest Points

Pattern 4: Sort + Pair Neighbors

Recognition:

TEXT

```
pairing
matching
minimize difference
maximize pairs
```

Sort first.

Examples:

TEXT

```
Assign Cookies
Boats to Save People
```

Pattern 5: Jump Game Style

Recognition:

TEXT

```
reach end
minimum jumps
farthest position
```

Examples:

- Jump Game
- Jump Game II

Thought Process

Keep track of:

```
PYTHON
farthest_reachable
```

and greedily extend the range.

Pattern 6: Huffman / Combine Smallest First

Recognition:

```
TEXT
merge costs
combine files
connect ropes
```

Examples:

- Minimum Cost to Connect Sticks

Use:

```
TEXT
min heap
```

Always merge the smallest first.

Pattern 7: Fractional Knapsack

Recognition:

```
TEXT
value / weight
ratio
profit density
```

Sort by:

```
TEXT
value / weight
```

descending.

Pattern 8: Monotonic Choice

Recognition:

TEXT

```
remove digits  
lexicographically smallest  
next greater  
stock span
```

Usually:

TEXT

```
monotonic stack
```

Examples:

- Remove K Digits
- Daily Temperatures
- Next Greater Element

The Greedy Checklist

When you see a problem, ask:

1. Can I sort?

Many greedy solutions start with:

PYTHON

```
arr.sort()
```

2. What local choice seems best?

Examples:

TEXT

```
earliest finishing interval  
largest profit  
smallest cost
```

3. If I make that choice, does it hurt future choices?

This is the crucial question.

Example:

TEXT

```
Interval Scheduling
```

Choosing earliest finish:

TEXT

helps future choices

Choosing latest finish:

TEXT

hurts future choices

Difference from DP

DP asks:

TEXT

What is the optimal answer for a subproblem?

Greedy asks:

TEXT

Can I make one choice now
and never regret it later?

Common Interview Greedy Problems

A good progression is:

These cover most of the recurring greedy patterns you'll encounter in interviews.

A useful mental model:

TEXT

DP = "Try all possibilities efficiently"

Greedy = "Find one choice that is always safe"

The skill in greedy problems is identifying **why that choice is safe**. Once you start solving interval scheduling, jump game, and heap-based greedy problems, you'll begin to see the same patterns repeat just like DP states repeat.

USER

What is the eligibility criteria for a problem to be solvable using greedy approach.

ASSISTANT

This is the central question in greedy algorithms.

Not every optimization problem can be solved greedily. For a problem to be solvable by a greedy approach, it typically needs **two properties**.

1. Greedy Choice Property

There must exist a **locally optimal choice** that can be made immediately and still lead to a globally optimal solution.

In other words:

TEXT

```
Make the best choice now
!"
Never need to reconsider it later
!"
Still get the optimal answer
```

Example: Interval Scheduling

Intervals:

TEXT

```
[1,3]
[2,5]
[4,6]
```

Greedy choice:

TEXT

```
Pick the interval that finishes earliest.
```

Why is it safe?

Because finishing earlier leaves the most room for future intervals.

Once chosen, we never regret it.

Counterexample: Coin Change

Coins:

TEXT

```
1, 3, 4
```

Target:

TEXT

```
6
```

Greedy:

TEXT

```
Take largest coin first
4 !' remaining 2
1 + 1
```

Uses:

TEXT

$4 + 1 + 1 = 3$ coins

Optimal:

TEXT

$3 + 3 = 2$ coins

Here the local choice is not safe.

So greedy fails.

2. Optimal Substructure

After making the greedy choice, the remaining problem must still be an optimal subproblem of the same form.

For example, in interval scheduling:

TEXT

Choose earliest finishing interval
!"
Now solve the same problem on the remaining intervals

The remainder is still an interval scheduling problem.

How to Test Whether Greedy Might Work

When you think of a greedy rule, ask:

Question 1

Can I commit to this choice permanently?

TEXT

If I choose this now,
will I ever need to undo it later?

If the answer is often "yes", greedy is suspicious.

Question 2

Can I prove that an optimal solution exists that makes this choice?

This is called an **exchange argument**.

Example

Suppose an optimal solution chooses interval A first.

You want to choose interval B first (earliest finish).

Can you replace A with B without making the solution worse?

If yes:

TEXT

```
Optimal solution
!"
Swap A for B
!"
Still optimal
```

Then B is a safe greedy choice.

This is the most common proof technique.

Practical Recognition

Greedy is often possible when:

Sorting reveals a natural order

TEXT

```
Intervals
Deadlines
Jobs
Meetings
```

Example:

TEXT

```
Sort by end time
Sort by start time
Sort by profit
```

Each decision affects only the future

Not the past.

Example:

TEXT

```
Jump Game
Interval Scheduling
Partition Labels
```

The best immediate choice clearly preserves future options

Example:

TEXT

```
Earliest finishing interval
```

leaves maximum future flexibility.

When Greedy Usually Fails

Be cautious when:

Multiple future consequences exist

Example:

TEXT

Longest Increasing Subsequence

Choosing the largest possible next element can destroy future possibilities.

You need to compare many combinations

Example:

TEXT

Knapsack

Subset Sum

Partition Equal Subset Sum

Different choices interact in complicated ways.

Usually DP is needed.

A Useful Rule of Thumb

When solving an optimization problem:

1. Try to identify a "best-looking" local choice.
2. Ask:

TEXT

Can I prove that every optimal solution can be modified to make this choice without becoming worse?

If yes, greedy is a strong candidate.

If not, you probably need DP, backtracking, or graph algorithms.

DP vs Greedy

DP says:

TEXT

I don't know which choice is best.

I'll explore all important possibilities.

Greedy says:

TEXT

I can prove this choice is always safe.

So I'll take it immediately.

That's the real eligibility criterion: **a provably safe local decision that never needs to be revisited.**

USER

```
class Solution: def minDistance(self, word1: str, word2: str) -> int: dp = [[0] * len(word2) for i in range(len(word1))] for i in range(len(word1)): dp[i][0] = i for j in range(len(word2)): dp[0][j] = j for i in range(len(word1)): for j in range(len(word2)): if word1[i] == word2[j]: dp[i][j] = dp[i - 1][j - 1] else: dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]) return dp[len(word1) - 1][len(word2) - 1]
```

ASSISTANT

This is the **Edit Distance** problem, and your code is very close, but there are several indexing issues.

First: What type of DP is this?

This is a **Two-Sequence DP** problem, just like LCS.

State:

PYTHON

```
dp[i][j]
```

means:

TEXT

```
Minimum operations needed to convert  
word1[0..i]  
into  
word2[0..j]
```

Problem 1: Wrong DP size

You created:

PYTHON

```
dp = [[0] * len(word2) for i in range(len(word1))]
```

But then you're accessing:

PYTHON

```
dp[i - 1][j]  
dp[i][j - 1]  
dp[i - 1][j - 1]
```

When `i=0` or `j=0`, Python uses negative indices:

PYTHON

```
dp[-1][j]
```

which is the last row, not what you want.

Standard Fix

Use an extra row and column:

```
PYTHON
dp = [[0] * (n + 1) for _ in range(m + 1)]
```

where

```
PYTHON
m = len(word1)
n = len(word2)
```

Problem 2: Base Cases

Suppose:

```
PYTHON
word1 = ""
word2 = "abc"
```

Need:

```
TEXT
3 insertions
```

Therefore:

```
PYTHON
dp[0][j] = j
```

Similarly:

```
PYTHON
dp[i][0] = i
```

because converting "abc" to "" needs 3 deletions.

Problem 3: Loop Bounds

With the extra row/column:

```
PYTHON
for i in range(1, m + 1):
    for j in range(1, n + 1):
```

Recurrence

If characters match:

PYTHON

```
word1[i-1] == word2[j-1]
```

No operation needed:

PYTHON

```
dp[i][j] = dp[i-1][j-1]
```

Otherwise we have three choices:

Delete

Delete current char from word1:

PYTHON

```
dp[i-1][j]
```

Insert

Insert current char from word2:

PYTHON

```
dp[i][j-1]
```

Replace

Replace current char:

PYTHON

```
dp[i-1][j-1]
```

Take the best:

PYTHON

```
dp[i][j] = 1 + min(  
    dp[i-1][j],  
    dp[i][j-1],  
    dp[i-1][j-1]  
)
```

Correct Code

PYTHON

```
class Solution:  
    def minDistance(self, word1: str, word2: str) -> int:  
  
        m = len(word1)  
        n = len(word2)
```

```

dp = [[0] * (n + 1) for _ in range(m + 1)]

for i in range(m + 1):
    dp[i][0] = i

for j in range(n + 1):
    dp[0][j] = j

for i in range(1, m + 1):
    for j in range(1, n + 1):

        if word1[i - 1] == word2[j - 1]:
            dp[i][j] = dp[i - 1][j - 1]
        else:
            dp[i][j] = 1 + min(
                dp[i - 1][j],      # delete
                dp[i][j - 1],      # insert
                dp[i - 1][j - 1]    # replace
            )

return dp[m][n]

```

Thought Process

For Two-Sequence DP, always ask:

What should I do with the last characters?

For:

PYTHON

`dp[i][j]`

we are converting:

TEXT

`word1[:i]`
`!`
`word2[:j]`

If last characters match:

TEXT

horse
 e

 house
 e

Ignore them and solve a smaller problem:

PYTHON

`dp[i-1][j-1]`

If they don't match, one operation must happen:

TEXT

Delete
Insert
Replace

Those three operations directly give the recurrence. This is the standard pattern for Edit Distance and many string-transformation DP problems.

USER

how to solve each pattern in dp

ASSISTANT

This is the right question to ask. Don't memorize 200 DP problems. Learn how to derive the solution for each **DP pattern**.

Here's a framework I recommend.

Step 0: Identify the DP Pattern

Before writing any recurrence, classify the problem.

Pattern	Typical State	Recognition
Prefix DP	$dp[i]$	One array/string, build left!right
Interval DP	$dp[i][j]$	Substring/subarray/range
Two-Sequence DP	$dp[i][j]$	Two strings/arrays
Grid DP	$dp[r][c]$	Matrix/path
Knapsack DP	$dp[i][target]$ or $dp[target]$	Choose items to reach target
Tree DP	$dp[node]$	Tree structure
Bitmask DP	$dp[mask]$	Small n , subsets/permutations

Once you know the pattern, the thought process becomes much easier.

1. Prefix DP

State

Ask:

What is the answer ending at or up to index i ?

Examples:

- LIS
- Decode Ways
- House Robber
- Maximum Product Subarray
- Word Break

Thought Process

Imagine you're standing at index `i`.

Ask:

How could I have arrived here?

Example: Decode Ways

At position `i`:

You either used

1 digit

or

2 digits

Therefore

```
dp[i]
!• dp[i - 1]
!• dp[i - 2]
```

General Template

Define `dp[i]`

For every possible previous state:
try transition

Take best/count/boolean

2. Interval DP

State

```
dp[i][j]
```

means

```
Answer for interval [i,j]
```

Thought Process

Always ask

What happens to the ends?

There are only a few possibilities.

Example:

Longest Palindromic Subsequence

```
a . . . . . a
```

If ends match

```
2 + inside
```

Else

```
discard left  
discard right
```

Example:

Burst Balloons

Instead of asking

```
Which balloon first?
```

Ask

```
Which balloon is burst LAST?
```

Changing the perspective makes the recurrence simple.

General template

```
Shrink interval  
  
left++  
right--
```

Split interval

k between i,j

3. Two-Sequence DP

State

`dp[i][j]`

means

Answer using

first i chars

first j chars

Thought process

Always ask

What happens to the last characters?

Only two cases.

Match

Take both.

No match

One of them must go.

Examples

LCS

match

1 + diagonal

no match

up

left

Edit Distance

replace

insert

delete

Distinct Subsequences

use character

don't use character

Almost every string DP follows this.

4. Grid DP

State

$dp[r][c]$

Thought process

Imagine standing in one cell.

Ask

Where could I have come from?

Example

Unique Paths

top

left

Minimum Path Sum

$\min(\text{top}, \text{left})$

Dungeon Game

Think backwards

Where must I go next?

5. Knapsack DP

State

$dp[i][\text{sum}]$

or

$dp[\text{sum}]$

Thought process

Ask

Do I take this item?

Two choices.

Take

Don't Take

Every recurrence becomes

`dp[i][target]`

=

take

or

don't take

Examples

Partition Equal Subset Sum

Target Sum

0/1 Knapsack

Coin Change

6. Tree DP

State

`dp[node]`

Thought process

Suppose I'm at one node.

Ask

What information do I need from my children?

Example

House Robber III

Need two states

rob node

don't rob node

because children depend on whether parent is robbed.

7. Bitmask DP

State

`dp[mask]`

Thought process

Mask tells

Which elements have already been used?

Transition

Choose one unused element.

Examples

Travelling Salesman

Assignment Problem

Shortest Hamiltonian Path

Universal DP Derivation Process

Whenever you see a DP problem, follow these **six questions**.

Question 1

What makes one subproblem different from another?

Examples

Index?

Interval?

Two indices?

Grid cell?

Subset?

Tree node?

That determines the state.

Question 2

Define the state in English.

Don't write code first.

Example

```
dp[i]
=
Longest increasing subsequence ending at i
```

or

```
dp[i][j]
=
LCS of first i chars and first j chars
```

Question 3

What is the last decision?

This is the most important question.

Examples

LIS

```
Which previous element comes before i?
```

Decode Ways

```
Did I use 1 digit or 2 digits?
```

LCS

```
What happened to the last characters?
```

Knapsack

```
Take or don't take?
```

Interval

```
What happened to the ends?
```

Question 4

Write the recurrence.

Question 5

Find the base cases.

Question 6

Determine the order to fill DP.

Examples

Prefix

l e f t ! ' r i g h t

Interval

s m a l l i n t e r v a l s ! ' l a r g e i n t e r v a l s

Grid

t o p - l e f t ! ' b o t t o m - r i g h t

LCS

s m a l l p r e f i x e s ! ' l a r g e p r e f i x e s

Tree

postorder DFS

A Summary You Can Memorize

PREFIX DP

State:

dp[i]

Question:

How do I reach i?

INTERVAL DP

State:

dp[i][j]

Question:

What happens to the ends?

TWO-SEQUENCE DP

State:

dp[i][j]

Question:

What happens to the last characters?

GRID DP

State:

dp[r][c]

Question:
Where did I come from?

KNAPSACK DP

State:
dp[i][sum]

Question:
Take or Don't Take?

TREE DP

State:
dp[node]

Question:
What do I need from my children?

BITMASK DP

State:
dp[mask]

Question:
Which elements are already used?

If you practice deriving solutions using these questions instead of memorizing recurrences, you'll find that many "new" DP problems are just variations of the same underlying patterns. The key skill isn't remembering formulas—it's recognizing the state and identifying the last decision that leads into that state.

USER

i heard another thing called paritition dp , is this in this list ?

ASSISTANT

Yes! **Partition DP** is absolutely a real DP category.

The reason I didn't list it separately is that it's often considered a **subtype of Interval DP**, but in competitive programming and advanced DP discussions it's usually taught as its own pattern because the recurrence is different.

What is Partition DP?

The defining idea is:

Split the problem into two independent parts at every possible position.

The recurrence almost always looks like:

PYTHON

```
dp[i][j] = best(
    dp[i][k] + dp[k+1][j]
)
```

or

PYTHON

```
dp[i][j] = best(
    dp[i][k] + dp[k][j]
)
```

where k is the partition point.

Notice the important difference:

You try every possible split.

Recognition Pattern

When you see questions like:

TEXT

Where should I split?
Where should I cut?
What is the best partition?

think:

TEXT

Partition DP

Example 1: Matrix Chain Multiplication (Classic)

Suppose matrices are:

TEXT

A B C D

To multiply all of them, where should you split?

Maybe:

TEXT

(A)(BCD)

or

TEXT

(AB)(CD)

or

TEXT

(ABC)(D)

Every split gives a different cost.

State:

PYTHON

dp[i][j]

=

TEXT

Minimum cost to multiply matrices i...j

Transition:

PYTHON

```
for k in range(i, j):
    dp[i][j] = min(
        dp[i][j],
        dp[i][k] + dp[k+1][j] + merge_cost
    )
```

Example 2: Burst Balloons

This is also Partition DP.

State:

PYTHON

dp[i][j]

=

TEXT

Maximum coins obtainable from interval [i,j]

Transition:

Choose the **last** balloon to burst.

PYTHON

```
for k in range(i, j+1):
    dp[i][j] = max(
        dp[i][k-1]
        +
        dp[k+1][j]
        +
        coins_from_last
    )
```

Again:

TEXT

Try every partition point k

Example 3: Minimum Cost to Cut a Stick

Cuts:

TEXT

0-----2-----5-----8-----10

Which cut should you perform first?

Try all of them.

State:

PYTHON

```
dp[i][j]
```

Transition:

PYTHON

```
for k in range(i+1, j):  
    ...
```

Again:

Partition DP.

How is it different from normal Interval DP?

Consider **Longest Palindromic Subsequence**.

Transition:

PYTHON

```
if s[i] == s[j]:  
    dp[i][j] = 2 + dp[i+1][j-1]  
else:  
    dp[i][j] = max(  
        dp[i+1][j],  
        dp[i][j-1]  
    )
```

Notice:

You never loop over all k .

You're only shrinking the interval from the ends.

Now consider Matrix Chain Multiplication.

PYTHON

```
for k in range(i, j):
```

You try:

TEXT

Split here

Split here

Split here

Split here...

That's Partition DP.

A Better Classification

Instead of saying there are only 6 DP patterns, I would organize them like this:

```
Dynamic Programming
%
% % %   P r e f i x   D P
%       d p [ i ]
%
% % %   I n t e r v a l   D P
%       %
%       % % %   S h r i n k i n g   I n t e r v a l   D P
%       %       L P S
%       %       L o n g e s t   P a l i n d r o m i c   S u b s t r i n g
%       %
%       % % %   P a r t i t i o n   D P
%               M a t r i x   C h a i n   M u l t i p l i c a t i o n
%               B u r s t   B a l l o o n s
%               M i n i m u m   C o s t   t o   C u t   S t i c k
%
% % %   T w o - S e q u e n c e   D P
%       L C S
%       E d i t   D i s t a n c e
%
% % %   G r i d   D P
%       U n i q u e   P a t h s
%
% % %   K n a p s a c k   D P
%       S u b s e t   S u m
%       C o i n   C h a n g e
%
% % %   T r e e   D P
%
% % %   B i t m a s k   D P
```

How to recognize Partition DP

Whenever you read a problem, ask:

Am I choosing one position `k` to divide the problem into two smaller independent subproblems?

If yes, it's almost certainly Partition DP.

The recurrence nearly always has this shape:

PYTHON

```
for k in range(i, j):
    dp[i][j] = best(
        dp[left part]
        +
        dp[right part]
        +
        cost_of_partition
    )
```

Rule of thumb

You can distinguish the two interval subtypes like this:

- **Shrinking Interval DP:** Ask "*What happens to the left end and right end?*" Examples: Longest Palindromic Subsequence, Longest Palindromic Substring.
- **Partition DP:** Ask "*Where should I split this interval?*" Examples: Matrix Chain Multiplication, Burst Balloons, Minimum Cost to Cut a Stick.

That's why Burst Balloons is often described as both an **Interval DP** problem (because the state is `dp[i][j]`) and a **Partition DP** problem (because the transition tries every split point `k`).

USER

Given an array `arr[]` which represents the dimensions of a sequence of matrices where the `i`th matrix has the dimensions `(arr[i-1] x arr[i])` for $i \geq 1$, find the most efficient way to multiply these matrices together. The efficient way is the one that involves the least number of multiplications.

Examples:

Input: `arr[] = [2, 1, 3, 4]` Output: 20 Explanation: There are 3 matrices of dimensions 2×1 , 1×3 , and 3×4 . Let this 3 input matrices be `M1`, `M2`, and `M3`. There are two ways to multiply: $((M1 \times M2) \times M3)$ and $(M1 \times (M2 \times M3))$, note that the result of $(M1 \times M2)$ is a 2×3 matrix and result of $(M2 \times M3)$ is a 1×4 matrix. $((M1 \times M2) \times M3)$ requires $(2 \times 1 \times 3) + (2 \times 3 \times 4) = 30$ $(M1 \times (M2 \times M3))$ requires $(1 \times 3 \times 4) + (2 \times 1 \times 4) = 20$. The minimum of these two is 20. teach me this pattern by solving this and show the thought process

ASSISTANT

Perfect. This is **the** classic Partition DP problem. If you understand this one, you'll recognize the pattern in Burst Balloons, Minimum Cost to Cut a Stick, Optimal BST, etc.

I'll teach it the same way we've been learning DP: **classify !' define state !' derive the recurrence naturally.**

Step 1: Understand the problem

Suppose

TEXT

`arr = [2,1,3,4]`

This represents 3 matrices:

TEXT

$M1 = 2 \times 1$

$M2 = 1 \times 3$

$M3 = 3 \times 4$

Notice something important:

The order of the matrices cannot change.

You **cannot** do

TEXT

$M3 \times M1$

You can only decide **where to put the parentheses**.

For example:

TEXT

$((M1M2)M3)$

or

$(M1(M2M3))$

So the real problem is:

Where should I split the multiplication?

That sentence is the biggest clue that this is **Partition DP**.

Step 2: Draw the recursion tree

Suppose we have

TEXT

$M1 \ M2 \ M3 \ M4$

What are all possible first splits?

TEXT

Split after $M1$

$(M1) \mid (M2 \ M3 \ M4)$

or

TEXT

Split after M2

(M1 M2) | (M3 M4)

or

TEXT

Split after M3

(M1 M2 M3) | (M4)

That's it.

Notice something:

We aren't deciding which matrix to multiply first.

We're deciding:

Where should the final multiplication happen?

Step 3: Define the subproblem

Whenever you see

TEXT

continuous matrices

think

TEXT

interval

Define

PYTHON

dp[i][j]

as

TEXT

Minimum cost to multiply
matrices $M_i \dots M_j$

Example:

TEXT

dp[2][4]

means

TEXT

Minimum cost to multiply

M2 M3 M4

Notice:

It doesn't care how.

Only the minimum cost.

Step 4: Think about the LAST multiplication

This is the key insight.

Suppose we want

TEXT

M1 M2 M3 M4

The last multiplication must combine **two already multiplied groups**.

Example:

TEXT

((M1M2)(M3M4))

The last multiplication is

TEXT

(M1M2)

×

(M3M4)

So before the last multiplication:

Left side has already been solved.

Right side has already been solved.

Therefore:

TEXT

Solve left interval

+

Solve right interval

+

Cost of multiplying the two results

That's the Partition DP idea.

Step 5: Where can the split occur?

Suppose

TEXT

$M_i \dots M_j$

Choose some split

TEXT

k

Then

TEXT

$M_i \dots M_k$

|

$M_{k+1} \dots M_j$

Left cost:

PYTHON

`dp[i][k]`

Right cost:

PYTHON

`dp[k+1][j]`

Now we need to multiply the resulting matrices.

Step 6: Cost of the final multiplication

This confuses almost everyone initially.

Let's derive it.

Suppose

TEXT

`arr = [10, 20, 30, 40]`

Matrices:

TEXT

$M_1 = 10 \times 20$

$M_2 = 20 \times 30$

$$M3 = 30 \times 40$$

Suppose we split after M2.

Left:

TEXT

M1M2

Dimensions?

TEXT

$$10 \times 30$$

Why?

Because

TEXT

$$10 \times 20$$

×

$$20 \times 30$$

=

$$10 \times 30$$

Similarly

Right:

TEXT

M3

is

TEXT

$$30 \times 40$$

Now multiply

TEXT

$$10 \times 30$$

×

$$30 \times 40$$

Cost:

TEXT

$$10 \times 30 \times 40$$

Where did these numbers come from?

Answer:

TEXT

`arr[i-1]`

`arr[k]`

`arr[j]`

Always.

So final multiplication cost is

PYTHON

```
arr[i-1] * arr[k] * arr[j]
```

Step 7: Build the recurrence

Try every possible split.

```
answer = "  
for every k:  
    left  
    +  
    right  
    +  
    final multiplication
```

So

PYTHON

```
dp[i][j] = min(  
    dp[i][k]  
    +  
    dp[k+1][j]  
    +  
    arr[i-1]*arr[k]*arr[j]  
)
```

This is **the** Partition DP recurrence.

Step 8: Base Case

Suppose

TEXT

```
i == j
```

Only one matrix.

Example

TEXT

M3

No multiplication required.

Cost:

TEXT

0

Therefore

PYTHON

```
dp[i][i] = 0
```

Step 9: Order of computation

Can we compute

PYTHON

```
dp[1][4]
```

immediately?

No.

Because it needs

TEXT

```
dp[1][2]
```

```
dp[3][4]
```

```
etc.
```

These are smaller intervals.

Therefore compute

TEXT

```
length = 1
```

```
length = 2
```

```
length = 3
```

```
...
```

Exactly like Interval DP.

Code

PYTHON

```
class Solution:

    def matrixMultiplication(self, arr):

        n = len(arr)

        dp = [[0] * n for _ in range(n)]

        for length in range(2, n):

            for i in range(1, n - length + 1):

                j = i + length - 1

                dp[i][j] = float('inf')

                for k in range(i, j):

                    cost = (
                        dp[i][k]
                        + dp[k + 1][j]
                        + arr[i - 1] * arr[k] * arr[j]
                    )

                    dp[i][j] = min(dp[i][j], cost)

        return dp[1][n - 1]
```

The Partition DP Recognition Pattern

Whenever you see a problem, ask these questions:

1. Is the answer for a contiguous interval?

TEXT

Yes

!“

Think `dp[i][j]`.

2. Am I trying to find the best place to split this interval?

TEXT

Yes

!“

It's probably Partition DP.

3. Does every split produce two independent subproblems?

```
[i.....k]
[k+1.....j]
```

!“

Still Partition DP.

4. Is there an extra cost for combining the two answers?

```
left
+
right
+
merge cost
```

!“

Classic Partition DP.

The Generic Partition DP Template

Almost every Partition DP problem follows this structure:

```
PYTHON
for length in range(...):
    for i in range(...):
        j = ...
        dp[i][j] = INF
        for k in range(i, j):
            dp[i][j] = min(
                dp[i][j],
                dp[i][k]          # left interval
                + dp[k+1][j]      # right interval
                + cost(i, k, j)   # cost to combine
            )
```

The only thing that changes from problem to problem is the `cost(i, k, j)` term.

This same template solves:

- **Matrix Chain Multiplication** !' `cost = arr[i-1] * arr[k] * arr[j]`
- **Burst Balloons** !' `cost = nums[i-1] * nums[k] * nums[j+1]`
- **Minimum Cost to Cut a Stick** !' `cost = cuts[j+1] - cuts[i-1]`

Once you recognize "choose the best split point `k`", you'll start seeing the same recurrence pattern across all of these

problems.

USER

first explain this questions , what is this cost ?

ASSISTANT

Good idea. Before DP, you should first understand **what the problem is asking**. The "cost" comes from **how matrix multiplication works**, not from DP.

What is a matrix?

A matrix is just a table of numbers.

Example:

A 2×3 matrix:

TEXT

```
1 2 3
4 5 6
```

It has:

- 2 rows
- 3 columns

So we write:

TEXT

2×3

When can two matrices be multiplied?

Suppose we have

TEXT

$A = 2 \times 3$

$B = 3 \times 4$

These **can** be multiplied because:

TEXT

columns of A = rows of B

$3 = 3$

The result has dimensions:

TEXT

$$2 \times 4$$

Rule:

TEXT

$$(a \times b) \times (b \times c)$$

!"

$$(a \times c)$$

What is the "cost" of multiplying matrices?

This is the key.

Suppose

TEXT

$$A = 2 \times 3$$

$$B = 3 \times 4$$

The result is

TEXT

$$2 \times 4$$

How many individual multiplications are required?

The formula is:

TEXT

$$\text{rows} \times \text{common_dimension} \times \text{columns}$$

or

TEXT

$$2 \times 3 \times 4 = 24$$

This number is called the **cost**.

It is **not** the value inside the matrix.

It is the **number of scalar multiplications** your computer performs.

Why is the cost $2 \times 3 \times 4$?

Let's compute one entry of the result.

The result is:

TEXT

2×4

So it has:

TEXT

8 entries

Take the first entry.

To compute it:

TEXT

(1×?)

+

(2×?)

+

(3×?)

You perform

TEXT

3 multiplications

Every entry needs **3 multiplications**.

There are

TEXT

$2 \times 4 = 8$

entries.

Therefore

TEXT

$8 \times 3 = 24$

which is

TEXT

$2 \times 3 \times 4$

Why does the parenthesization matter?

Now comes the interesting part.

Suppose

TEXT

arr = [2,1,3,4]

Matrices are

TEXT

$$M1 = 2 \times 1$$

$$M2 = 1 \times 3$$

$$M3 = 3 \times 4$$

There are two ways.

Option 1

Multiply

TEXT

$$(M1 \times M2)$$

first.

Cost:

TEXT

$$2 \times 1 \times 3 = 6$$

Resulting matrix:

TEXT

$$2 \times 3$$

Now multiply with M3.

TEXT

$$(2 \times 3) \times (3 \times 4)$$

Cost:

TEXT

$$2 \times 3 \times 4 = 24$$

Total:

TEXT

$$6 + 24 = 30$$

Option 2

Multiply

TEXT

$(M2 \times M3)$

first.

Cost:

TEXT

$1 \times 3 \times 4 = 12$

Result:

TEXT

1×4

Now multiply

TEXT

$(2 \times 1) \times (1 \times 4)$

Cost:

TEXT

$2 \times 1 \times 4 = 8$

Total:

TEXT

$12 + 8 = 20$

Notice something surprising.

The **same three matrices** are multiplied.

Yet one order costs

TEXT

30 multiplications

and the other costs

TEXT

20 multiplications.

That's exactly what this problem asks:

Where should I place the parentheses so that the total number of scalar multiplications is minimum?

Why don't the matrices change?

We are **not** changing the order.

This is **not allowed**:

TEXT

$M3 \times M1 \times M2$

The order must remain

TEXT

$M1 \ M2 \ M3$

The only thing we choose is:

TEXT

$((M1M2)M3)$

or

$(M1(M2M3))$

Summary

The **cost** is simply:

The number of scalar multiplications needed to multiply two matrices.

If you multiply

TEXT

$(a \times b) \times (b \times c)$

then the cost is

TEXT

$a \times b \times c$

The Matrix Chain Multiplication problem is about finding the **parenthesization** that minimizes the **sum of these multiplication costs**.

Once this intuition is clear, the DP recurrence becomes much more natural because every split computes:

- the cheapest way to multiply the left group,
- the cheapest way to multiply the right group,
- **plus** the cost of multiplying the two resulting matrices together.

USER

for k in range(i, j):

cost = (dp[i][k]

- `dp[k + 1][j]`
- `arr[i - 1] arr[k] arr[j]`

) this is confusing , i mean , you start i from 1 , for some reason you used `arr[i - 1]` instead of i

ASSISTANT

Excellent question. This is exactly the part that confuses everyone. The reason is that there are **two different numbering systems** being used.

There are two arrays

Suppose

TEXT

```
arr = [2, 1, 3, 4]
```

This is **not** an array of matrices.

It is an array of **dimensions**.

```
arr index:  0    1    2    3
            |    |    |    |
arr =       [2,  1,  3,  4]
```

From these dimensions, we create matrices:

```
M1 = arr[0] × arr[1] = 2 × 1
M2 = arr[1] × arr[2] = 1 × 3
M3 = arr[2] × arr[3] = 3 × 4
```

Notice the relationship:

Matrix	Rows	Columns
M1	<code>arr[0]</code>	<code>arr[1]</code>
M2	<code>arr[1]</code>	<code>arr[2]</code>
M3	<code>arr[2]</code>	<code>arr[3]</code>

So matrix `Mi` starts at ``arr[i-1]``, not `arr[i]`.

Why does DP start from 1?

We define

PYTHON

```
dp[i][j]
```

to mean

Minimum cost to multiply

$M_i \dots M_j$

So i and j are **matrix numbers**, not indices in `arr`.

For our example:

`dp[1][3]`

means

$M_1 \ M_2 \ M_3$

not

`arr[1] arr[2] arr[3]`

Now let's derive the formula

Suppose we are solving

`dp[1][3]`

That means:

$M_1 \ M_2 \ M_3$

Suppose we split after $k = 2$.

Left:

$M_1 \ M_2$

Right:

M_3

What are the dimensions of the left result?

Left contains

$M_1 = 2 \times 1$

$M_2 = 1 \times 3$

After multiplying them:

$(2 \times 1)(1 \times 3)$

```
!"
```

```
2x3
```

How do we get these dimensions from `arr` ?

Rows:

```
arr[0]
```

Columns:

```
arr[2]
```

Notice:

```
i = 1
```

```
k = 2
```

Rows =

```
arr[i-1]
```

Columns =

```
arr[k]
```

What about the right side?

Right is

```
M3
```

Dimensions:

```
3 x 4
```

Rows:

```
arr[2]
```

Columns:

```
arr[3]
```

Again,

Rows =

```
arr[k]
```

Columns =

```
arr[j]
```

because

```
j = 3
```

Now multiply the two results

Left result:

```
2 × 3
```

Right result:

```
3 × 4
```

Cost:

```
2 × 3 × 4
```

Now replace these numbers by their `arr` positions.

```
2 = arr[0] = arr[i-1]
```

```
3 = arr[2] = arr[k]
```

```
4 = arr[3] = arr[j]
```

Therefore:

```
PYTHON
```

```
arr[i-1] * arr[k] * arr[j]
```

A visual way to remember it

Imagine the dimensions as **points**:

```
arr:  2  -----  1  -----  3  -----  4
      ^         ^         ^         ^
      a0       a1       a2       a3
```

Matrices are the **edges** between consecutive points:

```

M1:  a0 ---- a1
M2:      a1 ---- a2
M3:      a2 ---- a3

```

If you solve

```
Mi ... Mj
```

then the resulting matrix always spans from

```
arr[i-1]
```

to

```
arr[j]
```

because:

- the **first matrix** contributes the number of rows (`arr[i-1]`)
- the **last matrix** contributes the number of columns (`arr[j]`)

If you split at `k`:

```
Mi ... Mk | Mk+1 ... Mj
```

then:

Left result:

```
arr[i-1] × arr[k]
```

Right result:

```
arr[k] × arr[j]
```

Multiplying these two costs:

```

(arr[i-1] × arr[k]) × (arr[k] × arr[j])
!
arr[i-1] * arr[k] * arr[j]

```

This `arr[i-1]` is **not an arbitrary trick**. It comes directly from the definition:

Matrix `Mi` has dimensions `arr[i-1] × arr[i]`.

Once that definition is fixed, every Matrix Chain Multiplication recurrence naturally uses `arr[i-1]`, `arr[k]`, and `arr[j]`.

USER

why not `dp[0][n-1]` ?

ASSISTANT

Another excellent question. This comes down to what your **state actually means**.

First, how many matrices are there?

Suppose

TEXT

```
arr = [2,1,3,4]
```

There are **4 dimensions**, but only **3 matrices**.

Dimensions:

```
2  ----  1  ----  3  ----  4
^         ^         ^         ^
0         1         2         3
```

Matrices are formed **between** consecutive dimensions:

```
M1 = 2 × 1
M2 = 1 × 3
M3 = 3 × 4
```

So:

- `len(arr) = 4`
- Number of matrices = `3 = len(arr) - 1`

What does `dp[i][j]` mean?

We defined:

PYTHON

```
dp[i][j]
```

to mean

Minimum cost to multiply matrices `Mi` through `Mj`.

Notice:

Matrices are numbered

M1
M2
M3

There is **no** `M0`.

So valid states are:

dp[1][1]
dp[1][2]
dp[2][3]
dp[1][3]

There is **no meaning** for

PYTHON
dp[0][3]

because

M0 doesn't exist.

Then why allocate an $n \times n$ table?

We do

PYTHON
n = len(arr)

dp = [[0] * n for _ in range(n)]

This creates indices

0 1 2 3

We simply **ignore row 0 and column 0**.

It's a convenience so that the matrix number **i** matches the DP index **i**.

You can think of it as:

	0	1	2	3
0	x	x	x	x
1			x	'
2			x	x
3			x	x

Only the shaded (') entries are meaningful.

Why return `dp[1][n-1]` ?

For

```
arr = [2,1,3,4]
```

we want to multiply

```
M1 M2 M3
```

The state representing **all matrices** is

```
dp[1][3]
```

Since

```
n = 4
```

that's

```
PYTHON
dp[1][n-1]
```

Could we define it differently?

Yes! Many books use **0-based matrix numbering** instead.

They define:

```
PYTHON
dp[i][j]
```

=

Minimum cost to multiply matrices $i..j$, where matrices are numbered 0 to $m-1$.

For our example:

```
Matrix 0 = 2x1
Matrix 1 = 1x3
Matrix 2 = 3x4
```

Now the answer would indeed be

```
PYTHON
dp[0][m-1]
```

where

```
m = number of matrices = len(arr)-1
```

In that version, the cost formula becomes:

PYTHON

```
arr[i] * arr[k+1] * arr[j+1]
```

because the indexing shifts by one.

Which version is better?

Both are correct.

The **1-based matrix numbering** is often used in textbooks because the matrix definition is naturally:

```
Matrix i = arr[i-1] × arr[i]
```

so the recurrence becomes:

PYTHON

```
arr[i-1] * arr[k] * arr[j]
```

The **0-based version** feels more natural in Python since it avoids wasting the 0th row and column.

Key takeaway

The choice of returning `dp[1][n-1]` is **not a property of the algorithm**. It's simply a consequence of how we **defined the state**. If you redefine the state with 0-based matrix indices, then the answer naturally becomes `dp[0][m-1]`, and the recurrence changes accordingly.

USER

i can't get the intuition

ASSISTANT

The intuition becomes much clearer if you **forget the code completely**.

The mistake almost everyone makes is trying to understand the recurrence before understanding **what the split actually represents**.

Let's build it from scratch.

Step 1: Forget DP

Suppose you have only **three matrices**.

TEXT

A = 10×20
B = 20×30
C = 30×40

Question:

How many ways are there to multiply them?

Only two.

$((AB)C)$

or

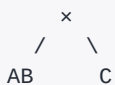
$(A(BC))$

Notice something.

You are **not choosing which multiplication happens first.**

You're choosing **where to put the last multiplication.**

Look at $((AB)C)$



Before the last multiplication:

- left side (AB) is already completely solved
- right side (C) is already solved

Now look at

$(A(BC))$



Again:

Before the last multiplication:

- left side is solved
- right side is solved

This is the key observation.

Every parenthesization ends with one final multiplication.

That final multiplication always combines

left interval
×
right interval

Step 2: Now take four matrices

A B C D

Ask yourself:

Where can the **last multiplication** happen?

Option 1

$(A) \times (BCD)$

Option 2

$(AB) \times (CD)$

Option 3

$(ABC) \times (D)$

Those are the **only possibilities**.

Notice what changed.

We're no longer thinking

"Which matrices should I multiply first?"

Instead we're thinking

"What are the two groups that are multiplied at the very end?"

That is exactly what **k** represents.

Step 3: What does **k** mean?

Suppose

A B C D

and

```
k = 2
```

means

```
A B | C D
```

Everything on the left must already be multiplied into one matrix.

Everything on the right must already be multiplied into one matrix.

Only then do we multiply those two results.

So

```
cost
=
cost(left)
+
cost(right)
+
cost(final multiplication)
```

This is the whole recurrence.

Step 4: Now let's understand the dimensions

Suppose

```
arr = [10, 20, 30, 40, 50]
```

Matrices are

```
M1 = 10×20
```

```
M2 = 20×30
```

```
M3 = 30×40
```

```
M4 = 40×50
```

Now suppose

```
dp[2][4]
```

means

```
M2 M3 M4
```

Suppose

$k = 3$

Split here

M2 M3 | M4

Question:

After multiplying

M2 M3

what are its dimensions?

M2

20×30

M3

30×40

Result

20×40

Notice something.

Rows came from

M2

Columns came from

M3

The middle dimensions disappeared.

This is always true.

Now look at the indices.

Rows

20

=

arr[1]

=

```
arr[i-1]
```

Columns

```
40  
=  
arr[3]  
=  
arr[k]
```

So the left result is always

```
arr[i-1] × arr[k]
```

Right side

```
M4
```

Dimensions

```
40×50
```

Rows

```
arr[3]  
=  
arr[k]
```

Columns

```
arr[4]  
=  
arr[j]
```

So the right result is always

```
arr[k] × arr[j]
```

Now multiply these two results.

```
(arr[i-1] × arr[k])  
×
```

```
(arr[k] × arr[j])
```

Cost is

```
arr[i-1]
×
arr[k]
×
arr[j]
```

This is exactly where

```
PYTHON
arr[i-1] * arr[k] * arr[j]
```

comes from.

The intuition behind `dp[1][n-1]`

Imagine the matrices are numbered like this:

```
arr = [10,20,30,40]

      ! '      ! '      ! '      ! '
    d0 d1 d2 d3
```

Matrices live **between** dimensions.

```
d0 ---- d1 ---- d2 ---- d3
      M1      M2      M3
```

Notice:

There is

```
M1
M2
M3
```

There is **no M0**.

So asking

```
dp[0][3]
```

would literally mean

Multiply

M0

through

M3

But **M0 doesn't exist**.

That's why, in this formulation, we start from **1**.

One thing that usually makes it click

Think of `arr` as **fence posts** and matrices as **fence panels**.

Posts:

| --- | --- | --- | --- |

0 1 2 3

Panels (matrices) are between posts.

| --M1-- | --M2-- | --M3-- |

The posts are indexed `0..3`, but the panels are `1..3`.

`dp` talks about **panels**, not posts.

So `dp[1][3]` means "multiply panels M1 through M3."

If you're comfortable with this now, I can also show you a **0-based version** of Matrix Chain Multiplication (`dp[0][m-1]`), which many people find much more intuitive in Python because it avoids the `i-1` indexing entirely.